JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Arrays

java.lang.Object
    java.util.Arrays

```
public final class Arrays
extends Object
```

This class contains various methods for manipulating arrays (such as sorting and searching). This class also contains a static factory that allows arrays to be viewed as lists.

The methods in this class all throw a `NullPointerException`, if the specified array reference is null, except where noted.

The documentation for the methods contained in this class includes brief descriptions of the *implementations*. Such descriptions should be regarded as *implementation notes*, rather than parts of the *specification*. Implementors should feel free to substitute other algorithms, so long as the specification itself is adhered to. (For example, the algorithm used by `sort(Object[])` does not have to be a MergeSort, but it does have to be *stable*.)

This class is a member of the Java Collections Framework.

Since:

1.2

Method Summary

All Methods	Static Methods	
Modifier and Type	**Method**	**Description**
`static <T> List<T>`	`asList(T... a)`	Returns a fixed-size list backed by the specified array.
`static int`	`binarySearch(byte[] a, byte key)`	Searches the specified array of bytes for the specified value using the binary search algorithm.
`static int`	`binarySearch(byte[] a, int fromIndex, int toIndex, byte key)`	Searches a range of the specified array of bytes for the specified value using the binary search algorithm.
`static int`		

	binarySearch(char[] a, char key)	Searches the specified array of chars for the specified value using the binary search algorithm.
static int	**binarySearch**(char[] a, int fromIndex, int toIndex, char key)	Searches a range of the specified array of chars for the specified value using the binary search algorithm.
static int	**binarySearch** (double[] a, double key)	Searches the specified array of doubles for the specified value using the binary search algorithm.
static int	**binarySearch** (double[] a, int fromIndex, int toIndex, double key)	Searches a range of the specified array of doubles for the specified value using the binary search algorithm.
static int	**binarySearch**(float[] a, float key)	Searches the specified array of floats for the specified value using the binary search algorithm.
static int	**binarySearch**(float[] a, int fromIndex, int toIndex, float key)	Searches a range of the specified array of floats for the specified value using the binary search algorithm.
static int	**binarySearch**(int[] a, int key)	Searches the specified array of ints for the specified value using the binary search algorithm.
static int	**binarySearch**(int[] a, int fromIndex, int toIndex, int key)	Searches a range of the specified array of ints for the specified value using the binary search algorithm.
static int		Searches a range of the specified array of longs

	binarySearch(long[] a, int fromIndex, int toIndex, long key)	for the specified value using the binary search algorithm.
static int	**binarySearch**(long[] a, long key)	Searches the specified array of longs for the specified value using the binary search algorithm.
static int	**binarySearch**(short[] a, int fromIndex, int toIndex, short key)	Searches a range of the specified array of shorts for the specified value using the binary search algorithm.
static int	**binarySearch**(short[] a, short key)	Searches the specified array of shorts for the specified value using the binary search algorithm.
static int	**binarySearch**(Object[] a, int fromIndex, int toIndex, Object key)	Searches a range of the specified array for the specified object using the binary search algorithm.
static int	**binarySearch**(Object[] a, Object key)	Searches the specified array for the specified object using the binary search algorithm.
static <T> int	**binarySearch**(T[] a, int fromIndex, int toIndex, T key, Comparator<? super T> c)	Searches a range of the specified array for the specified object using the binary search algorithm.
static <T> int	**binarySearch**(T[] a, T key, Comparator<? super T> c)	Searches the specified array for the specified object using the binary search algorithm.
static int	**compare**(boolean[] a, boolean[] b)	Compares two boolean arrays lexicographically.

static int	compare(boolean[] a, int aFromIndex, int aToIndex, boolean[] b, int bFromIndex, int bToIndex)	Compares two boolean arrays lexicographically over the specified ranges.
static int	compare(byte[] a, byte[] b)	Compares two byte arrays lexicographically.
static int	compare(byte[] a, int aFromIndex, int aToIndex, byte[] b, int bFromIndex, int bToIndex)	Compares two byte arrays lexicographically over the specified ranges.
static int	compare(char[] a, char[] b)	Compares two char arrays lexicographically.
static int	compare(char[] a, int aFromIndex, int aToIndex, char[] b, int bFromIndex, int bToIndex)	Compares two char arrays lexicographically over the specified ranges.
static int	compare(double[] a, double[] b)	Compares two double arrays lexicographically.
static int	compare(double[] a, int aFromIndex, int aToIndex, double[] b, int bFromIndex, int bToIndex)	Compares two double arrays lexicographically over the specified ranges.
static int	compare(float[] a, float[] b)	Compares two float arrays lexicographically.
static int	compare(float[] a, int aFromIndex, int aToIndex, float[] b, int bFromIndex, int bToIndex)	Compares two float arrays lexicographically over the specified ranges.
static int		

	compare(int[] a, int[] b)	Compares two int arrays lexicographically.
static int	**compare**(int[] a, int aFromIndex, int aToIndex, int[] b, int bFromIndex, int bToIndex)	Compares two int arrays lexicographically over the specified ranges.
static int	**compare**(long[] a, int aFromIndex, int aToIndex, long[] b, int bFromIndex, int bToIndex)	Compares two long arrays lexicographically over the specified ranges.
static int	**compare**(long[] a, long[] b)	Compares two long arrays lexicographically.
static int	**compare**(short[] a, int aFromIndex, int aToIndex, short[] b, int bFromIndex, int bToIndex)	Compares two short arrays lexicographically over the specified ranges.
static int	**compare**(short[] a, short[] b)	Compares two short arrays lexicographically.
static <T extends Comparable<? super T>> int	**compare**(T[] a, int aFromIndex, int aToIndex, T[] b, int bFromIndex, int bToIndex)	Compares two Object arrays lexicographically over the specified ranges.
static <T> int	**compare**(T[] a, int aFromIndex, int aToIndex, T[] b, int bFromIndex, int bToIndex, Comparator<? super T> cmp)	Compares two Object arrays lexicographically over the specified ranges.
static <T extends Comparable<? super T>> int	**compare**(T[] a, T[] b)	Compares two Object arrays, within comparable elements, lexicographically.

static <T> int	**compare**(T[] a, T[] b, Comparator<? super T> cmp)	Compares two Object arrays lexicographically using a specified comparator.
static int	**compareUnsigned** (byte[] a, byte[] b)	Compares two byte arrays lexicographically, numerically treating elements as unsigned.
static int	**compareUnsigned** (byte[] a, int aFromIndex, int aToIndex, byte[] b, int bFromIndex, int bToIndex)	Compares two byte arrays lexicographically over the specified ranges, numerically treating elements as unsigned.
static int	**compareUnsigned** (int[] a, int[] b)	Compares two int arrays lexicographically, numerically treating elements as unsigned.
static int	**compareUnsigned** (int[] a, int aFromIndex, int aToIndex, int[] b, int bFromIndex, int bToIndex)	Compares two int arrays lexicographically over the specified ranges, numerically treating elements as unsigned.
static int	**compareUnsigned** (long[] a, int aFromIndex, int aToIndex, long[] b, int bFromIndex, int bToIndex)	Compares two long arrays lexicographically over the specified ranges, numerically treating elements as unsigned.
static int	**compareUnsigned** (long[] a, long[] b)	Compares two long arrays lexicographically, numerically treating elements as unsigned.
static int	**compareUnsigned** (short[] a, int aFromIndex, int aToIndex, short[] b,	Compares two short arrays lexicographically over the specified ranges, numerically

	`int bFromIndex, int bToIndex)`	treating elements as unsigned.
`static int`	**`compareUnsigned`** `(short[] a, short[] b)`	Compares two `short` arrays lexicographically, numerically treating elements as unsigned.
`static boolean[]`	**`copyOf`** `(boolean[] original, int newLength)`	Copies the specified array, truncating or padding with `false` (if necessary) so the copy has the specified length.
`static byte[]`	**`copyOf`**`(byte[] original, int newLength)`	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
`static char[]`	**`copyOf`**`(char[] original, int newLength)`	Copies the specified array, truncating or padding with null characters (if necessary) so the copy has the specified length.
`static double[]`	**`copyOf`** `(double[] original, int newLength)`	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
`static float[]`	**`copyOf`** `(float[] original, int newLength)`	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
`static int[]`	**`copyOf`**`(int[] original, int newLength)`	Copies the specified array, truncating or

		padding with zeros (if necessary) so the copy has the specified length.
static long[]	**copyOf**(long[] original, int newLength)	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
static short[]	**copyOf**(short[] original, int newLength)	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
static <T> T[]	**copyOf**(T[] original, int newLength)	Copies the specified array, truncating or padding with nulls (if necessary) so the copy has the specified length.
static <T,U> T[]	**copyOf**(U[] original, int newLength, Class<? extends T[]> newType)	Copies the specified array, truncating or padding with nulls (if necessary) so the copy has the specified length.
static boolean[]	**copyOfRange**(boolean[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static byte[]	**copyOfRange**(byte[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static char[]	**copyOfRange**(char[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static double[]	**copyOfRange**(double[] original, int from, int to)	Copies the specified range of the specified array into a new array.

static float[]	**copyOfRange**(float[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static int[]	**copyOfRange**(int[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static long[]	**copyOfRange**(long[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static short[]	**copyOfRange**(short[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static <T> T[]	**copyOfRange**(T[] original, int from, int to)	Copies the specified range of the specified array into a new array.
static <T,U> T[]	**copyOfRange**(U[] original, int from, int to, Class<? extends T[]> newType)	Copies the specified range of the specified array into a new array.
static boolean	**deepEquals**(Object[] a1, Object[] a2)	Returns `true` if the two specified arrays are *deeply equal* to one another.
static int	**deepHashCode**(Object[] a)	Returns a hash code based on the "deep contents" of the specified array.
static String	**deepToString**(Object[] a)	Returns a string representation of the "deep contents" of the specified array.
static boolean	**equals**(boolean[] a, boolean[] a2)	Returns `true` if the two specified arrays of booleans are *equal* to one another.
static boolean		

	equals(boolean[] a, int aFromIndex, int aToIndex, boolean[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of booleans, over the specified ranges, are *equal* to one another.
static boolean	**equals**(byte[] a, byte[] a2)	Returns true if the two specified arrays of bytes are *equal* to one another.
static boolean	**equals**(byte[] a, int aFromIndex, int aToIndex, byte[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of bytes, over the specified ranges, are *equal* to one another.
static boolean	**equals**(char[] a, char[] a2)	Returns true if the two specified arrays of chars are *equal* to one another.
static boolean	**equals**(char[] a, int aFromIndex, int aToIndex, char[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of chars, over the specified ranges, are *equal* to one another.
static boolean	**equals**(double[] a, double[] a2)	Returns true if the two specified arrays of doubles are *equal* to one another.
static boolean	**equals**(double[] a, int aFromIndex, int aToIndex, double[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of doubles, over the specified ranges, are *equal* to one another.
static boolean	**equals**(float[] a, float[] a2)	Returns true if the two specified arrays of floats are *equal* to one another.
static boolean	**equals**(float[] a, int aFromIndex, int aToIndex,	Returns true if the two specified arrays of floats, over the specified

	float[] b, int bFromIndex, int bToIndex)	ranges, are *equal* to one another.
static boolean	**equals**(int[] a, int[] a2)	Returns true if the two specified arrays of ints are *equal* to one another.
static boolean	**equals**(int[] a, int aFromIndex, int aToIndex, int[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of ints, over the specified ranges, are *equal* to one another.
static boolean	**equals**(long[] a, int aFromIndex, int aToIndex, long[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of longs, over the specified ranges, are *equal* to one another.
static boolean	**equals**(long[] a, long[] a2)	Returns true if the two specified arrays of longs are *equal* to one another.
static boolean	**equals**(short[] a, int aFromIndex, int aToIndex, short[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of shorts, over the specified ranges, are *equal* to one another.
static boolean	**equals**(short[] a, short[] a2)	Returns true if the two specified arrays of shorts are *equal* to one another.
static boolean	**equals**(Object[] a, int aFromIndex, int aToIndex, Object[] b, int bFromIndex, int bToIndex)	Returns true if the two specified arrays of Objects, over the specified ranges, are *equal* to one another.
static boolean		

	equals(Object[] a, Object[] a2)	Returns true if the two specified arrays of Objects are *equal* to one another.
static <T> boolean	**equals**(T[] a, int aFromIndex, int aToIndex, T[] b, int bFromIndex, int bToIndex, Comparator<? super T> cmp)	Returns true if the two specified arrays of Objects, over the specified ranges, are *equal* to one another.
static <T> boolean	**equals**(T[] a, T[] a2, Comparator<? super T> cmp)	Returns true if the two specified arrays of Objects are *equal* to one another.
static void	**fill**(boolean[] a, boolean val)	Assigns the specified boolean value to each element of the specified array of booleans.
static void	**fill**(boolean[] a, int fromIndex, int toIndex, boolean val)	Assigns the specified boolean value to each element of the specified range of the specified array of booleans.
static void	**fill**(byte[] a, byte val)	Assigns the specified byte value to each element of the specified array of bytes.
static void	**fill**(byte[] a, int fromIndex, int toIndex, byte val)	Assigns the specified byte value to each element of the specified range of the specified array of bytes.
static void	**fill**(char[] a, char val)	Assigns the specified char value to each element of the specified array of chars.
static void		

	`fill(char[] a, int fromIndex, int toIndex, char val)`	Assigns the specified char value to each element of the specified range of the specified array of chars.
`static void`	`fill(double[] a, double val)`	Assigns the specified double value to each element of the specified array of doubles.
`static void`	`fill(double[] a, int fromIndex, int toIndex, double val)`	Assigns the specified double value to each element of the specified range of the specified array of doubles.
`static void`	`fill(float[] a, float val)`	Assigns the specified float value to each element of the specified array of floats.
`static void`	`fill(float[] a, int fromIndex, int toIndex, float val)`	Assigns the specified float value to each element of the specified range of the specified array of floats.
`static void`	`fill(int[] a, int val)`	Assigns the specified int value to each element of the specified array of ints.
`static void`	`fill(int[] a, int fromIndex, int toIndex, int val)`	Assigns the specified int value to each element of the specified range of the specified array of ints.
`static void`	`fill(long[] a, int fromIndex, int toIndex, long val)`	Assigns the specified long value to each element of the specified range of the specified array of longs.
`static void`		

	fill(long[] a, long val)	Assigns the specified long value to each element of the specified array of longs.
static void	**fill**(short[] a, int fromIndex, int toIndex, short val)	Assigns the specified short value to each element of the specified range of the specified array of shorts.
static void	**fill**(short[] a, short val)	Assigns the specified short value to each element of the specified array of shorts.
static void	**fill**(Object[] a, int fromIndex, int toIndex, Object val)	Assigns the specified Object reference to each element of the specified range of the specified array of Objects.
static void	**fill**(Object[] a, Object val)	Assigns the specified Object reference to each element of the specified array of Objects.
static int	**hashCode**(boolean[] a)	Returns a hash code based on the contents of the specified array.
static int	**hashCode**(byte[] a)	Returns a hash code based on the contents of the specified array.
static int	**hashCode**(char[] a)	Returns a hash code based on the contents of the specified array.
static int	**hashCode**(double[] a)	Returns a hash code based on the contents of the specified array.
static int	**hashCode**(float[] a)	Returns a hash code based on the contents of the specified array.

static int	**hashCode**(int[] a)	Returns a hash code based on the contents of the specified array.
static int	**hashCode**(long[] a)	Returns a hash code based on the contents of the specified array.
static int	**hashCode**(short[] a)	Returns a hash code based on the contents of the specified array.
static int	**hashCode**(Object[] a)	Returns a hash code based on the contents of the specified array.
static int	**mismatch**(boolean[] a, boolean[] b)	Finds and returns the index of the first mismatch between two boolean arrays, otherwise return -1 if no mismatch is found.
static int	**mismatch**(boolean[] a, int aFromIndex, int aToIndex, boolean[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two boolean arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int	**mismatch**(byte[] a, byte[] b)	Finds and returns the index of the first mismatch between two byte arrays, otherwise return -1 if no mismatch is found.
static int	**mismatch**(byte[] a, int aFromIndex, int aToIndex, byte[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two byte arrays over the specified ranges, otherwise return -1 if no mismatch is found.

static int	**mismatch**(char[] a, char[] b)	Finds and returns the index of the first mismatch between two `char` arrays, otherwise return -1 if no mismatch is found.
static int	**mismatch**(char[] a, int aFromIndex, int aToIndex, char[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two `char` arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int	**mismatch**(double[] a, double[] b)	Finds and returns the index of the first mismatch between two `double` arrays, otherwise return -1 if no mismatch is found.
static int	**mismatch**(double[] a, int aFromIndex, int aToIndex, double[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two `double` arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int	**mismatch**(float[] a, float[] b)	Finds and returns the index of the first mismatch between two `float` arrays, otherwise return -1 if no mismatch is found.
static int	**mismatch**(float[] a, int aFromIndex, int aToIndex, float[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two `float` arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int		

	mismatch(int[] a, int[] b)	Finds and returns the index of the first mismatch between two `int` arrays, otherwise return -1 if no mismatch is found.
static int	**mismatch**(int[] a, int aFromIndex, int aToIndex, int[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two `int` arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int	**mismatch**(long[] a, int aFromIndex, int aToIndex, long[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two `long` arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int	**mismatch**(long[] a, long[] b)	Finds and returns the index of the first mismatch between two `long` arrays, otherwise return -1 if no mismatch is found.
static int	**mismatch**(short[] a, int aFromIndex, int aToIndex, short[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two `short` arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int	**mismatch**(short[] a, short[] b)	Finds and returns the index of the first mismatch between two `short` arrays, otherwise return -1 if no mismatch is found.
static int		

	mismatch(Object[] a, int aFromIndex, int aToIndex, Object[] b, int bFromIndex, int bToIndex)	Finds and returns the relative index of the first mismatch between two Object arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static int	**mismatch**(Object[] a, Object[] b)	Finds and returns the index of the first mismatch between two Object arrays, otherwise return -1 if no mismatch is found.
static <T> int	**mismatch**(T[] a, int aFromIndex, int aToIndex, T[] b, int bFromIndex, int bToIndex, Comparator<? super T> cmp)	Finds and returns the relative index of the first mismatch between two Object arrays over the specified ranges, otherwise return -1 if no mismatch is found.
static <T> int	**mismatch**(T[] a, T[] b, Comparator<? super T> cmp)	Finds and returns the index of the first mismatch between two Object arrays, otherwise return -1 if no mismatch is found.
static void	**parallelPrefix**(double[] array, int fromIndex, int toIndex, DoubleBinaryOperator op)	Performs parallelPrefix(double[], DoubleBinaryOperator) for the given subrange of the array.
static void	**parallelPrefix**(double[] array, DoubleBinaryOperator op)	Cumulates, in parallel, each element of the given array in place, using the supplied function.
static void	**parallelPrefix**(int[] array, int fromIndex,	Performs parallelPrefix(int[], IntBinaryOperator) for

	int toIndex, IntBinaryOperator op)	the given subrange of the array.
static void	**parallelPrefix** (int[] array, IntBinaryOperator op)	Cumulates, in parallel, each element of the given array in place, using the supplied function.
static void	**parallelPrefix** (long[] array, int fromIndex, int toIndex, LongBinaryOperator op)	Performs parallelPrefix(long[], LongBinaryOperator) for the given subrange of the array.
static void	**parallelPrefix** (long[] array, LongBinaryOperator op)	Cumulates, in parallel, each element of the given array in place, using the supplied function.
static <T> void	**parallelPrefix** (T[] array, int fromIndex, int toIndex, BinaryOperator<T> op)	Performs parallelPrefix(Object[], BinaryOperator) for the given subrange of the array.
static <T> void	**parallelPrefix** (T[] array, BinaryOperator<T> op)	Cumulates, in parallel, each element of the given array in place, using the supplied function.
static void	**parallelSetAll** (double[] array, IntToDoubleFunction gene	Set all elements of the specified array, in parallel, using the provided generator function to compute each element.
static void	**parallelSetAll** (int[] array, IntUnaryOperator generat	Set all elements of the specified array, in parallel, using the

		provided generator function to compute each element.
static void	**parallelSetAll**(long[] array, IntToLongFunction genera	Set all elements of the specified array, in parallel, using the provided generator function to compute each element.
static <T> void	**parallelSetAll**(T[] array, IntFunction<? extends T> generator)	Set all elements of the specified array, in parallel, using the provided generator function to compute each element.
static void	**parallelSort**(byte[] a)	Sorts the specified array into ascending numerical order.
static void	**parallelSort**(byte[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending numerical order.
static void	**parallelSort**(char[] a)	Sorts the specified array into ascending numerical order.
static void	**parallelSort**(char[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending numerical order.
static void	**parallelSort**(double[] a)	Sorts the specified array into ascending numerical order.
static void	**parallelSort**(double[] a,	Sorts the specified range of the array into

	int fromIndex, int toIndex)	ascending numerical order.
static void	**parallelSort**(float[] a)	Sorts the specified array into ascending numerical order.
static void	**parallelSort**(float[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending numerical order.
static void	**parallelSort**(int[] a)	Sorts the specified array into ascending numerical order.
static void	**parallelSort**(int[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending numerical order.
static void	**parallelSort**(long[] a)	Sorts the specified array into ascending numerical order.
static void	**parallelSort**(long[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending numerical order.
static void	**parallelSort**(short[] a)	Sorts the specified array into ascending numerical order.
static void	**parallelSort**(short[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending numerical order.
static <T extends Comparable<? super T>> void	**parallelSort**(T[] a)	Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.

static <T extends Comparable<? super T>> void	**parallelSort**(T[] a, int fromIndex, int toIndex)	Sorts the specified range of the specified array of objects into ascending order, according to the natural ordering of its elements.
static <T> void	**parallelSort**(T[] a, int fromIndex, int toIndex, Comparator<? super T> cmp)	Sorts the specified range of the specified array of objects according to the order induced by the specified comparator.
static <T> void	**parallelSort**(T[] a, Comparator<? super T> cmp)	Sorts the specified array of objects according to the order induced by the specified comparator.
static void	**setAll**(double[] array, IntToDoubleFunction gene	Set all elements of the specified array, using the provided generator function to compute each element.
static void	**setAll**(int[] array, IntUnaryOperator generat	Set all elements of the specified array, using the provided generator function to compute each element.
static void	**setAll**(long[] array, IntToLongFunction genera	Set all elements of the specified array, using the provided generator function to compute each element.
static <T> void	**setAll**(T[] array, IntFunction<? extends T> generator)	Set all elements of the specified array, using the provided generator function to compute each element.
static void	**sort**(byte[] a)	Sorts the specified array into ascending numerical order.

static void	sort(byte[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending order.
static void	sort(char[] a)	Sorts the specified array into ascending numerical order.
static void	sort(char[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending order.
static void	sort(double[] a)	Sorts the specified array into ascending numerical order.
static void	sort(double[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending order.
static void	sort(float[] a)	Sorts the specified array into ascending numerical order.
static void	sort(float[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending order.
static void	sort(int[] a)	Sorts the specified array into ascending numerical order.
static void	sort(int[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending order.
static void	sort(long[] a)	Sorts the specified array into ascending numerical order.
static void	sort(long[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending order.
static void	sort(short[] a)	Sorts the specified array into ascending numerical order.

static void	**sort**(short[] a, int fromIndex, int toIndex)	Sorts the specified range of the array into ascending order.
static void	**sort**(Object[] a)	Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.
static void	**sort**(Object[] a, int fromIndex, int toIndex)	Sorts the specified range of the specified array of objects into ascending order, according to the natural ordering of its elements.
static <T> void	**sort**(T[] a, int fromIndex, int toIndex, Comparator<? super T> c)	Sorts the specified range of the specified array of objects according to the order induced by the specified comparator.
static <T> void	**sort**(T[] a, Comparator<? super T> c)	Sorts the specified array of objects according to the order induced by the specified comparator.
static Spliterator.OfDouble	**spliterator**(double[] array)	Returns a Spliterator.OfDouble covering all of the specified array.
static Spliterator.OfDouble	**spliterator**(double[] array, int startInclusive, int endExclusive)	Returns a Spliterator.OfDouble covering the specified range of the specified array.
static Spliterator.OfInt	**spliterator**(int[] array)	Returns a Spliterator.OfInt covering all of the specified array.
static Spliterator.OfInt	**spliterator**(int[] array,	Returns a Spliterator.OfInt

	int startInclusive, int endExclusive)	covering the specified range of the specified array.
static Spliterator.OfLong	**spliterator** (long[] array)	Returns a Spliterator.OfLong covering all of the specified array.
static Spliterator.OfLong	**spliterator** (long[] array, int startInclusive, int endExclusive)	Returns a Spliterator.OfLong covering the specified range of the specified array.
static <T> Spliterator<T>	**spliterator**(T[] array)	Returns a Spliterator covering all of the specified array.
static <T> Spliterator<T>	**spliterator**(T[] array, int startInclusive, int endExclusive)	Returns a Spliterator covering the specified range of the specified array.
static DoubleStream	**stream**(double[] array)	Returns a sequential DoubleStream with the specified array as its source.
static DoubleStream	**stream**(double[] array, int startInclusive, int endExclusive)	Returns a sequential DoubleStream with the specified range of the specified array as its source.
static IntStream	**stream**(int[] array)	Returns a sequential IntStream with the specified array as its source.
static IntStream	**stream**(int[] array, int startInclusive, int endExclusive)	Returns a sequential IntStream with the specified range of the

		specified array as its source.
static LongStream	**stream**(long[] array)	Returns a sequential LongStream with the specified array as its source.
static LongStream	**stream**(long[] array, int startInclusive, int endExclusive)	Returns a sequential LongStream with the specified range of the specified array as its source.
static <T> Stream <T>	**stream**(T[] array)	Returns a sequential Stream with the specified array as its source.
static <T> Stream <T>	**stream**(T[] array, int startInclusive, int endExclusive)	Returns a sequential Stream with the specified range of the specified array as its source.
static String	**toString**(boolean[] a)	Returns a string representation of the contents of the specified array.
static String	**toString**(byte[] a)	Returns a string representation of the contents of the specified array.
static String	**toString**(char[] a)	Returns a string representation of the contents of the specified array.
static String	**toString**(double[] a)	Returns a string representation of the contents of the specified array.

static String	**toString**(float[] a)	Returns a string representation of the contents of the specified array.
static String	**toString**(int[] a)	Returns a string representation of the contents of the specified array.
static String	**toString**(long[] a)	Returns a string representation of the contents of the specified array.
static String	**toString**(short[] a)	Returns a string representation of the contents of the specified array.
static String	**toString**(Object[] a)	Returns a string representation of the contents of the specified array.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Details

sort

```
public static void sort(int[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

a - the array to be sorted

sort

```
public static void sort(int[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the array into ascending order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

sort

```
public static void sort(long[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

sort

```
public static void sort(long[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the array into ascending order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

sort

```
public static void sort(short[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

sort

```
public static void sort(short[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the array into ascending order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

sort

```
public static void sort(char[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

sort

```
public static void sort(char[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the array into ascending order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

a - the array to be sorted

fromIndex - the index of the first element, inclusive, to be sorted

toIndex - the index of the last element, exclusive, to be sorted

Throws:

IllegalArgumentException - if fromIndex > toIndex

ArrayIndexOutOfBoundsException - if fromIndex < 0 or toIndex > a.length

sort

```
public static void sort(byte[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

a - the array to be sorted

sort

```
public static void sort(byte[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the array into ascending order. The range to be sorted extends from the index fromIndex, inclusive, to the index toIndex, exclusive. If fromIndex == toIndex, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

a - the array to be sorted

fromIndex - the index of the first element, inclusive, to be sorted

toIndex - the index of the last element, exclusive, to be sorted

Throws:

IllegalArgumentException - if fromIndex > toIndex

ArrayIndexOutOfBoundsException - if fromIndex < 0 or toIndex > a.length

sort

```
public static void sort(float[] a)
```

Sorts the specified array into ascending numerical order.

The < relation does not provide a total order on all float values: -0.0f == 0.0f is true and a Float.NaN value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method Float.compareTo(Float): -0.0f is treated as less than value 0.0f and Float.NaN is considered greater than any other value and all Float.NaN values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

a - the array to be sorted

sort

```
public static void sort(float[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the array into ascending order. The range to be sorted extends from the index fromIndex, inclusive, to the index toIndex, exclusive. If fromIndex == toIndex, the range to be sorted is empty.

The < relation does not provide a total order on all float values: -0.0f == 0.0f is true and a Float.NaN value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method Float.compareTo(Float): -0.0f is treated as less than value 0.0f and Float.NaN is considered greater than any other value and all Float.NaN values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

sort

```
public static void sort(double[] a)
```

Sorts the specified array into ascending numerical order.

The < relation does not provide a total order on all double values: `-0.0d == 0.0d` is `true` and a `Double.NaN` value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method `Double.compareTo(Double)`: `-0.0d` is treated as less than value `0.0d` and `Double.NaN` is considered greater than any other value and all `Double.NaN` values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

sort

```
public static void sort(double[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the array into ascending order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

The < relation does not provide a total order on all double values: `-0.0d == 0.0d` is `true` and a `Double.NaN` value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method `Double.compareTo(Double)`: `-0.0d` is treated as less than value

`0.0d` and `Double.NaN` is considered greater than any other value and all `Double.NaN` values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

parallelSort

```
public static void parallelSort(byte[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

Since:

1.8

parallelSort

```
public static void parallelSort(byte[] a,
                                int fromIndex,
                                int toIndex)
```

Sorts the specified range of the array into ascending numerical order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

Since:

1.8

parallelSort

```
public static void parallelSort(char[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

Since:

1.8

parallelSort

```
public static void parallelSort(char[] a,
                                int fromIndex,
                                int toIndex)
```

Sorts the specified range of the array into ascending numerical order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

Since:

1.8

parallelSort

```
public static void parallelSort(short[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

Since:

1.8

parallelSort

```
public static void parallelSort(short[] a,
                                int fromIndex,
                                int toIndex)
```

Sorts the specified range of the array into ascending numerical order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

Since:

1.8

parallelSort

```
public static void parallelSort(int[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

Since:

1.8

parallelSort

```
public static void parallelSort(int[] a,
                                int fromIndex,
                                int toIndex)
```

Sorts the specified range of the array into ascending numerical order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

Since:

1.8

parallelSort

```
public static void parallelSort(long[] a)
```

Sorts the specified array into ascending numerical order.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

Since:

1.8

parallelSort

```
public static void parallelSort(long[] a,
                                int fromIndex,
                                int toIndex)
```

Sorts the specified range of the array into ascending numerical order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

Since:

1.8

parallelSort

```
public static void parallelSort(float[] a)
```

Sorts the specified array into ascending numerical order.

The < relation does not provide a total order on all float values: `-0.0f == 0.0f` is `true` and a `Float.NaN` value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method `Float.compareTo(Float)`: `-0.0f` is treated as less than value `0.0f` and `Float.NaN` is considered greater than any other value and all `Float.NaN` values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

a - the array to be sorted

Since:

1.8

parallelSort

```
public static void parallelSort(float[] a,
                                int fromIndex,
                                int toIndex)
```

Sorts the specified range of the array into ascending numerical order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

The < relation does not provide a total order on all float values: `-0.0f == 0.0f` is `true` and a `Float.NaN` value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method `Float.compareTo(Float)`: `-0.0f` is treated as less than value `0.0f` and `Float.NaN` is considered greater than any other value and all `Float.NaN` values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

Since:

1.8

parallelSort

```
public static void parallelSort(double[] a)
```

Sorts the specified array into ascending numerical order.

The < relation does not provide a total order on all double values: `-0.0d == 0.0d` is `true` and a `Double.NaN` value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method `Double.compareTo(Double)`: `-0.0d` is treated as less than value

`0.0d` and `Double.NaN` is considered greater than any other value and all `Double.NaN` values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

Since:

1.8

parallelSort

```
public static void parallelSort(double[] a,
                                int fromIndex,
                                int toIndex)
```

Sorts the specified range of the array into ascending numerical order. The range to be sorted extends from the index `fromIndex`, inclusive, to the index `toIndex`, exclusive. If `fromIndex == toIndex`, the range to be sorted is empty.

The < relation does not provide a total order on all double values: `-0.0d == 0.0d` is `true` and a `Double.NaN` value compares neither less than, greater than, nor equal to any value, even itself. This method uses the total order imposed by the method `Double.compareTo(Double)`: `-0.0d` is treated as less than value `0.0d` and `Double.NaN` is considered greater than any other value and all `Double.NaN` values are considered equal.

Implementation Note:

The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley and Josh Bloch. This algorithm offers O(n log(n)) performance on all data sets, and is typically faster than traditional (one-pivot) Quicksort implementations.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element, inclusive, to be sorted

`toIndex` - the index of the last element, exclusive, to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

Since:

1.8

parallelSort

```
public static <T extends Comparable<? super T>>
void parallelSort(T[] a)
```

Sorts the specified array of objects into ascending order, according to the natural ordering of its elements. All elements in the array must implement the `Comparable` interface. Furthermore, all elements in the array must be *mutually comparable* (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the array).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation Note:

The sorting algorithm is a parallel sort-merge that breaks the array into sub-arrays that are themselves sorted and then merged. When the sub-array length reaches a minimum granularity, the sub-array is sorted using the appropriate `Arrays.sort` method. If the length of the specified array is less than the minimum granularity, then it is sorted using the appropriate `Arrays.sort` method. The algorithm requires a working space no greater than the size of the original array. The `ForkJoin common pool` is used to execute any parallel tasks.

Type Parameters:

T - the class of the objects to be sorted

Parameters:

a - the array to be sorted

Throws:

`ClassCastException` - if the array contains elements that are not *mutually comparable* (for example, strings and integers)

`IllegalArgumentException` - (optional) if the natural ordering of the array elements is found to violate the `Comparable` contract

Since:

1.8

parallelSort

```
public static <T extends Comparable<? super T>>
void parallelSort(T[] a,
                  int fromIndex,
                  int toIndex)
```

Sorts the specified range of the specified array of objects into ascending order, according to the natural ordering of its elements. The range to be sorted extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be sorted is empty.) All elements in this range must implement the `Comparable` interface. Furthermore, all elements in this range must be *mutually comparable* (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the array).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation Note:

The sorting algorithm is a parallel sort-merge that breaks the array into sub-arrays that are themselves sorted and then merged. When the sub-array length reaches a minimum granularity, the sub-array is sorted using the appropriate `Arrays.sort` method. If the length of the specified array is less than the minimum granularity, then it is sorted using the appropriate `Arrays.sort` method. The algorithm requires a working space no greater than the size of the specified range of the original array. The `ForkJoin common pool` is used to execute any parallel tasks.

Type Parameters:

`T` - the class of the objects to be sorted

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element (inclusive) to be sorted

`toIndex` - the index of the last element (exclusive) to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex` or (optional) if the natural ordering of the array elements is found to violate the `Comparable` contract

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

`ClassCastException` - if the array contains elements that are not *mutually comparable* (for example, strings and integers).

Since:

1.8

parallelSort

```
public static <T> void parallelSort(T[] a,
                                    Comparator<? super T> cmp)
```

Sorts the specified array of objects according to the order induced by the specified comparator. All elements in the array must be *mutually comparable* by the specified comparator (that is, `c.compare(e1, e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the array).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation Note:

The sorting algorithm is a parallel sort-merge that breaks the array into sub-arrays that are themselves sorted and then merged. When the sub-array length reaches a minimum granularity, the sub-array is sorted using the appropriate `Arrays.sort` method. If the length of the specified array is less than the minimum granularity, then it is sorted using the appropriate `Arrays.sort` method. The algorithm requires a working space no greater than the size of the original array. The `ForkJoin common pool` is used to execute any parallel tasks.

Type Parameters:

`T` - the class of the objects to be sorted

Parameters:

`a` - the array to be sorted

`cmp` - the comparator to determine the order of the array. A `null` value indicates that the elements' natural ordering should be used.

Throws:

`ClassCastException` - if the array contains elements that are not *mutually comparable* using the specified comparator

`IllegalArgumentException` - (optional) if the comparator is found to violate the `Comparator` contract

Since:

1.8

parallelSort

```
public static <T> void parallelSort(T[] a,
                                    int fromIndex,
                                    int toIndex,
                                    Comparator<? super T> cmp)
```

Sorts the specified range of the specified array of objects according to the order induced by the specified comparator. The range to be sorted extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be sorted is empty.) All elements in the range must be *mutually comparable* by the specified comparator (that is,

`c.compare(e1, e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the range).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation Note:

The sorting algorithm is a parallel sort-merge that breaks the array into sub-arrays that are themselves sorted and then merged. When the sub-array length reaches a minimum granularity, the sub-array is sorted using the appropriate `Arrays.sort` method. If the length of the specified array is less than the minimum granularity, then it is sorted using the appropriate `Arrays.sort` method. The algorithm requires a working space no greater than the size of the specified range of the original array. The `ForkJoin common pool` is used to execute any parallel tasks.

Type Parameters:

`T` - the class of the objects to be sorted

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element (inclusive) to be sorted

`toIndex` - the index of the last element (exclusive) to be sorted

`cmp` - the comparator to determine the order of the array. A `null` value indicates that the elements' natural ordering should be used.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex` or (optional) if the natural ordering of the array elements is found to violate the `Comparable` contract

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

`ClassCastException` - if the array contains elements that are not *mutually comparable* (for example, strings and integers).

Since:

1.8

sort

```
public static void sort(Object[] a)
```

Sorts the specified array of objects into ascending order, according to the natural ordering of its elements. All elements in the array must implement the `Comparable` interface. Furthermore, all elements in the array must be

mutually comparable (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the array).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation note: This implementation is a stable, adaptive, iterative mergesort that requires far fewer than n lg(n) comparisons when the input array is partially sorted, while offering the performance of a traditional mergesort when the input array is randomly ordered. If the input array is nearly sorted, the implementation requires approximately n comparisons. Temporary storage requirements vary from a small constant for nearly sorted input arrays to n/2 object references for randomly ordered input arrays.

The implementation takes equal advantage of ascending and descending order in its input array, and can take advantage of ascending and descending order in different parts of the same input array. It is well-suited to merging two or more sorted arrays: simply concatenate the arrays and sort the resulting array.

The implementation was adapted from Tim Peters's list sort for Python (TimSort). It uses techniques from Peter McIlroy's "Optimistic Sorting and Information Theoretic Complexity", in Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, pp 467-474, January 1993.

Parameters:

`a` - the array to be sorted

Throws:

`ClassCastException` - if the array contains elements that are not *mutually comparable* (for example, strings and integers)

`IllegalArgumentException` - (optional) if the natural ordering of the array elements is found to violate the `Comparable` contract

sort

```
public static void sort(Object[] a,
                        int fromIndex,
                        int toIndex)
```

Sorts the specified range of the specified array of objects into ascending order, according to the natural ordering of its elements. The range to be sorted extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be sorted is empty.) All elements in this range must implement the `Comparable` interface. Furthermore, all elements in this range must be *mutually comparable* (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the array).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation note: This implementation is a stable, adaptive, iterative mergesort that requires far fewer than n lg(n) comparisons when the input array is partially sorted, while offering the performance of a traditional mergesort when the input array is randomly ordered. If the input array is nearly sorted, the implementation requires approximately n comparisons. Temporary storage requirements vary from a small constant for nearly sorted input arrays to n/2 object references for randomly ordered input arrays.

The implementation takes equal advantage of ascending and descending order in its input array, and can take advantage of ascending and descending order in different parts of the same input array. It is well-suited to merging two or more sorted arrays: simply concatenate the arrays and sort the resulting array.

The implementation was adapted from Tim Peters's list sort for Python (TimSort). It uses techniques from Peter McIlroy's "Optimistic Sorting and Information Theoretic Complexity", in Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, pp 467-474, January 1993.

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element (inclusive) to be sorted

`toIndex` - the index of the last element (exclusive) to be sorted

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex` or (optional) if the natural ordering of the array elements is found to violate the `Comparable` contract

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

`ClassCastException` - if the array contains elements that are not *mutually comparable* (for example, strings and integers).

sort

```
public static <T> void sort(T[] a,
                            Comparator<? super T> c)
```

Sorts the specified array of objects according to the order induced by the specified comparator. All elements in the array must be *mutually comparable* by the specified comparator (that is, `c.compare(e1, e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the array).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation note: This implementation is a stable, adaptive, iterative mergesort that requires far fewer than n lg(n) comparisons when the input array is partially sorted, while offering the performance of a traditional mergesort when the input array is randomly ordered. If the input array is nearly sorted, the implementation requires approximately n comparisons. Temporary storage requirements vary from a small constant for nearly sorted input arrays to n/2 object references for randomly ordered input arrays.

The implementation takes equal advantage of ascending and descending order in its input array, and can take advantage of ascending and descending order in different parts of the same input array. It is well-suited to merging two or more sorted arrays: simply concatenate the arrays and sort the resulting array.

The implementation was adapted from Tim Peters's list sort for Python (TimSort). It uses techniques from Peter McIlroy's "Optimistic Sorting and Information Theoretic Complexity", in Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, pp 467-474, January 1993.

Type Parameters:

T - the class of the objects to be sorted

Parameters:

a - the array to be sorted

c - the comparator to determine the order of the array. A `null` value indicates that the elements' natural ordering should be used.

Throws:

`ClassCastException` - if the array contains elements that are not *mutually comparable* using the specified comparator

`IllegalArgumentException` - (optional) if the comparator is found to violate the `Comparator` contract

sort

```
public static <T> void sort(T[] a,
                            int fromIndex,
                            int toIndex,
                            Comparator<? super T> c)
```

Sorts the specified range of the specified array of objects according to the order induced by the specified comparator. The range to be sorted extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be sorted is empty.) All elements in the

range must be *mutually comparable* by the specified comparator (that is, `c.compare(e1, e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the range).

This sort is guaranteed to be *stable*: equal elements will not be reordered as a result of the sort.

Implementation note: This implementation is a stable, adaptive, iterative mergesort that requires far fewer than n lg(n) comparisons when the input array is partially sorted, while offering the performance of a traditional mergesort when the input array is randomly ordered. If the input array is nearly sorted, the implementation requires approximately n comparisons. Temporary storage requirements vary from a small constant for nearly sorted input arrays to n/2 object references for randomly ordered input arrays.

The implementation takes equal advantage of ascending and descending order in its input array, and can take advantage of ascending and descending order in different parts of the same input array. It is well-suited to merging two or more sorted arrays: simply concatenate the arrays and sort the resulting array.

The implementation was adapted from Tim Peters's list sort for Python (TimSort). It uses techniques from Peter McIlroy's "Optimistic Sorting and Information Theoretic Complexity", in Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, pp 467-474, January 1993.

Type Parameters:

`T` - the class of the objects to be sorted

Parameters:

`a` - the array to be sorted

`fromIndex` - the index of the first element (inclusive) to be sorted

`toIndex` - the index of the last element (exclusive) to be sorted

`c` - the comparator to determine the order of the array. A `null` value indicates that the elements' natural ordering should be used.

Throws:

`ClassCastException` - if the array contains elements that are not *mutually comparable* using the specified comparator.

`IllegalArgumentException` - if `fromIndex > toIndex` or (optional) if the comparator is found to violate the `Comparator` contract

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

parallelPrefix

```
public static <T> void parallelPrefix(T[] array,
                                      BinaryOperator<T> op)
```

Cumulates, in parallel, each element of the given array in place, using the supplied function. For example if the array initially holds `[2, 1, 0, 3]` and the operation performs addition, then upon return the array holds `[2, 3, 3, 6]`. Parallel prefix computation is usually more efficient than sequential loops for large arrays.

Type Parameters:

`T` - the class of the objects in the array

Parameters:

`array` - the array, which is modified in-place by this method

`op` - a side-effect-free, associative function to perform the cumulation

Throws:

`NullPointerException` - if the specified array or function is null

Since:

1.8

parallelPrefix

```
public static <T> void parallelPrefix(T[] array,
                                      int fromIndex,
                                      int toIndex,
                                      BinaryOperator<T> op)
```

Performs `parallelPrefix(Object[], BinaryOperator)` for the given subrange of the array.

Type Parameters:

`T` - the class of the objects in the array

Parameters:

`array` - the array

`fromIndex` - the index of the first element, inclusive

`toIndex` - the index of the last element, exclusive

`op` - a side-effect-free, associative function to perform the cumulation

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > array.length`

NullPointerException - if the specified array or function is null

Since:

1.8

parallelPrefix

```
public static void parallelPrefix(long[] array,
                                  LongBinaryOperator op)
```

Cumulates, in parallel, each element of the given array in place, using the supplied function. For example if the array initially holds `[2, 1, 0, 3]` and the operation performs addition, then upon return the array holds `[2, 3, 3, 6]`. Parallel prefix computation is usually more efficient than sequential loops for large arrays.

Parameters:

`array` - the array, which is modified in-place by this method

`op` - a side-effect-free, associative function to perform the cumulation

Throws:

NullPointerException - if the specified array or function is null

Since:

1.8

parallelPrefix

```
public static void parallelPrefix(long[] array,
                                  int fromIndex,
                                  int toIndex,
                                  LongBinaryOperator op)
```

Performs `parallelPrefix(long[], LongBinaryOperator)` for the given subrange of the array.

Parameters:

`array` - the array

`fromIndex` - the index of the first element, inclusive

`toIndex` - the index of the last element, exclusive

`op` - a side-effect-free, associative function to perform the cumulation

Throws:

IllegalArgumentException - if `fromIndex > toIndex`

ArrayIndexOutOfBoundsException - if `fromIndex < 0` or `toIndex > array.length`

`NullPointerException` - if the specified array or function is null

Since:

1.8

parallelPrefix

```
public static void parallelPrefix(double[] array,
                                  DoubleBinaryOperator op)
```

Cumulates, in parallel, each element of the given array in place, using the supplied function. For example if the array initially holds `[2.0, 1.0, 0.0, 3.0]` and the operation performs addition, then upon return the array holds `[2.0, 3.0, 3.0, 6.0]`. Parallel prefix computation is usually more efficient than sequential loops for large arrays.

Because floating-point operations may not be strictly associative, the returned result may not be identical to the value that would be obtained if the operation was performed sequentially.

Parameters:

`array` - the array, which is modified in-place by this method

`op` - a side-effect-free function to perform the cumulation

Throws:

`NullPointerException` - if the specified array or function is null

Since:

1.8

parallelPrefix

```
public static void parallelPrefix(double[] array,
                                  int fromIndex,
                                  int toIndex,
                                  DoubleBinaryOperator op)
```

Performs `parallelPrefix(double[], DoubleBinaryOperator)` for the given subrange of the array.

Parameters:

`array` - the array

`fromIndex` - the index of the first element, inclusive

`toIndex` - the index of the last element, exclusive

`op` - a side-effect-free, associative function to perform the cumulation

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > array.length`

`NullPointerException` - if the specified array or function is null

Since:

1.8

parallelPrefix

```
public static void parallelPrefix(int[] array,
                                  IntBinaryOperator op)
```

Cumulates, in parallel, each element of the given array in place, using the supplied function. For example if the array initially holds `[2, 1, 0, 3]` and the operation performs addition, then upon return the array holds `[2, 3, 3, 6]`. Parallel prefix computation is usually more efficient than sequential loops for large arrays.

Parameters:

`array` - the array, which is modified in-place by this method

`op` - a side-effect-free, associative function to perform the cumulation

Throws:

`NullPointerException` - if the specified array or function is null

Since:

1.8

parallelPrefix

```
public static void parallelPrefix(int[] array,
                                  int fromIndex,
                                  int toIndex,
                                  IntBinaryOperator op)
```

Performs `parallelPrefix(int[], IntBinaryOperator)` for the given subrange of the array.

Parameters:

`array` - the array

`fromIndex` - the index of the first element, inclusive

`toIndex` - the index of the last element, exclusive

`op` - a side-effect-free, associative function to perform the cumulation

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > array.length`

`NullPointerException` - if the specified array or function is null

Since:

1.8

binarySearch

```
public static int binarySearch(long[] a,
                               long key)
```

Searches the specified array of longs for the specified value using the binary search algorithm. The array must be sorted (as by the `sort(long[])` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

binarySearch

```
public static int binarySearch(long[] a,
                               int fromIndex,
                               int toIndex,
                               long key)
```

Searches a range of the specified array of longs for the specified value using the binary search algorithm. The range must be sorted (as by the `sort(long[], int, int)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

binarySearch

```
public static int binarySearch(int[] a,
                               int key)
```

Searches the specified array of ints for the specified value using the binary search algorithm. The array must be sorted (as by the `sort(int[])` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

binarySearch

```
public static int binarySearch(int[] a,
                               int fromIndex,
                               int toIndex,
                               int key)
```

Searches a range of the specified array of ints for the specified value using the binary search algorithm. The range must be sorted (as by the `sort(int[], int, int)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

binarySearch

```
public static int binarySearch(short[] a,
                               short key)
```

Searches the specified array of shorts for the specified value using the binary search algorithm. The array must be sorted (as by the `sort(short[])` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

binarySearch

```
public static int binarySearch(short[] a,
                               int fromIndex,
                               int toIndex,
                               short key)
```

Searches a range of the specified array of shorts for the specified value using the binary search algorithm. The range must be sorted (as by the `sort(short[], int, int)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

binarySearch

```
public static int binarySearch(char[] a,
                               char key)
```

Searches the specified array of chars for the specified value using the binary search algorithm. The array must be sorted (as by the `sort(char[])` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

binarySearch

```
public static int binarySearch(char[] a,
                               int fromIndex,
                               int toIndex,
                               char key)
```

Searches a range of the specified array of chars for the specified value using the binary search algorithm. The range must be sorted (as by the `sort(char[], int, int)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of

the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

binarySearch

```
public static int binarySearch(byte[] a,
                               byte key)
```

Searches the specified array of bytes for the specified value using the binary search algorithm. The array must be sorted (as by the `sort(byte[])` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

a - the array to be searched

key - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

binarySearch

```
public static int binarySearch(byte[] a,
                               int fromIndex,
                               int toIndex,
                               byte key)
```

Searches a range of the specified array of bytes for the specified value using the binary search algorithm. The range must be sorted (as by the `sort(byte[], int, int)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements with the specified value, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

binarySearch

```
public static int binarySearch(double[] a,
                               double key)
```

Searches the specified array of doubles for the specified value using the binary search algorithm. The array must be sorted (as by the `sort(double[])` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found. This method considers all NaN values to be equivalent and equal.

Parameters:

`a` - the array to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

binarySearch

```
public static int binarySearch(double[] a,
                               int fromIndex,
                               int toIndex,
                               double key)
```

Searches a range of the specified array of doubles for the specified value using the binary search algorithm. The range must be sorted (as by the `sort(double[], int, int)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements with the specified value, there is no guarantee which one will be found. This method considers all NaN values to be equivalent and equal.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

binarySearch

```
public static int binarySearch(float[] a,
                               float key)
```

Searches the specified array of floats for the specified value using the binary search algorithm. The array must be sorted (as by the `sort(float[])` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no

guarantee which one will be found. This method considers all NaN values to be equivalent and equal.

Parameters:

`a` - the array to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

binarySearch

```
public static int binarySearch(float[] a,
                               int fromIndex,
                               int toIndex,
                               float key)
```

Searches a range of the specified array of floats for the specified value using the binary search algorithm. The range must be sorted (as by the `sort(float[], int, int)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements with the specified value, there is no guarantee which one will be found. This method considers all NaN values to be equivalent and equal.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

ArrayIndexOutOfBoundsException - if fromIndex < 0 or toIndex > a.length

Since:

1.6

binarySearch

```
public static int binarySearch(Object[] a,
                               Object key)
```

Searches the specified array for the specified object using the binary search algorithm. The array must be sorted into ascending order according to the natural ordering of its elements (as by the sort(Object[]) method) prior to making this call. If it is not sorted, the results are undefined. (If the array contains elements that are not mutually comparable (for example, strings and integers), it *cannot* be sorted according to the natural ordering of its elements, hence results are undefined.) If the array contains multiple elements equal to the specified object, there is no guarantee which one will be found.

Parameters:

a - the array to be searched

key - the value to be searched for

Returns:

index of the search key, if it is contained in the array; otherwise, (-(*insertion point*) - 1). The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or a.length if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

ClassCastException - if the search key is not comparable to the elements of the array.

binarySearch

```
public static int binarySearch(Object[] a,
                               int fromIndex,
                               int toIndex,
                               Object key)
```

Searches a range of the specified array for the specified object using the binary search algorithm. The range must be sorted into ascending order according to the natural ordering of its elements (as by the sort(Object[], int, int) method) prior to making this call. If it is not sorted, the results are

undefined. (If the range contains elements that are not mutually comparable (for example, strings and integers), it *cannot* be sorted according to the natural ordering of its elements, hence results are undefined.) If the range contains multiple elements equal to the specified object, there is no guarantee which one will be found.

Parameters:

`a` - the array to be searched

`fromIndex` - the index of the first element (inclusive) to be searched

`toIndex` - the index of the last element (exclusive) to be searched

`key` - the value to be searched for

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`ClassCastException` - if the search key is not comparable to the elements of the array within the specified range.

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

binarySearch

```
public static <T> int binarySearch(T[] a,
                                   T key,
                                   Comparator<? super T> c)
```

Searches the specified array for the specified object using the binary search algorithm. The array must be sorted into ascending order according to the specified comparator (as by the `sort(T[], Comparator)` method) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements equal to the specified object, there is no guarantee which one will be found.

Type Parameters:

`T` - the class of the objects in the array

Parameters:

a - the array to be searched

key - the value to be searched for

c - the comparator by which the array is ordered. A `null` value indicates that the elements' natural ordering should be used.

Returns:

index of the search key, if it is contained in the array; otherwise, `(-(*insertion point*) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or `a.length` if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`ClassCastException` - if the array contains elements that are not *mutually comparable* using the specified comparator, or the search key is not comparable to the elements of the array using this comparator.

binarySearch

```
public static <T> int binarySearch(T[] a,
                                   int fromIndex,
                                   int toIndex,
                                   T key,
                                   Comparator<? super T> c)
```

Searches a range of the specified array for the specified object using the binary search algorithm. The range must be sorted into ascending order according to the specified comparator (as by the `sort(T[], int, int, Comparator)` method) prior to making this call. If it is not sorted, the results are undefined. If the range contains multiple elements equal to the specified object, there is no guarantee which one will be found.

Type Parameters:

T - the class of the objects in the array

Parameters:

a - the array to be searched

fromIndex - the index of the first element (inclusive) to be searched

toIndex - the index of the last element (exclusive) to be searched

key - the value to be searched for

c - the comparator by which the array is ordered. A `null` value indicates that the elements' natural ordering should be used.

Returns:

index of the search key, if it is contained in the array within the specified range; otherwise, `(-(`*`insertion point`*`) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the array: the index of the first element in the range greater than the key, or `toIndex` if all elements in the range are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`ClassCastException` - if the range contains elements that are not *mutually comparable* using the specified comparator, or the search key is not comparable to the elements in the range using this comparator.

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0 or toIndex > a.length`

Since:

1.6

equals

```
public static boolean equals(long[] a,
                             long[] a2)
```

Returns `true` if the two specified arrays of longs are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

equals

```
public static boolean equals(long[] a,
                             int aFromIndex,
                             int aToIndex,
                             long[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of longs, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

equals

```
public static boolean equals(int[] a,
                             int[] a2)
```

Returns `true` if the two specified arrays of ints are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

equals

```
public static boolean equals(int[] a,
                             int aFromIndex,
                             int aToIndex,
                             int[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of ints, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

equals

```
public static boolean equals(short[] a,
                             short[] a2)
```

Returns `true` if the two specified arrays of shorts are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

equals

```
public static boolean equals(short[] a,
                             int aFromIndex,
                             int aToIndex,
                             short[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of shorts, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

equals

```
public static boolean equals(char[] a,
                             char[] a2)
```

Returns `true` if the two specified arrays of chars are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

equals

```
public static boolean equals(char[] a,
                             int aFromIndex,
                             int aToIndex,
                             char[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of chars, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

equals

```
public static boolean equals(byte[] a,
                             byte[] a2)
```

Returns `true` if the two specified arrays of bytes are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

equals

```
public static boolean equals(byte[] a,
                             int aFromIndex,
                             int aToIndex,
                             byte[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of bytes, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

equals

```
public static boolean equals(boolean[] a,
                             boolean[] a2)
```

Returns `true` if the two specified arrays of booleans are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

equals

```
public static boolean equals(boolean[] a,
                             int aFromIndex,
                             int aToIndex,
                             boolean[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of booleans, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

equals

```
public static boolean equals(double[] a,
                             double[] a2)
```

Returns `true` if the two specified arrays of doubles are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`. Two doubles `d1` and `d2` are considered equal if:

```
    Double.valueOf(d1).equals(Double.valueOf(d2))
```

(Unlike the == operator, this method considers `NaN` equal to itself, and 0.0d unequal to -0.0d.)

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

See Also:

`Double.equals(Object)`

equals

```
public static boolean equals(double[] a,
                             int aFromIndex,
                             int aToIndex,
                             double[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of doubles, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Two doubles `d1` and `d2` are considered equal if:

```
    Double.valueOf(d1).equals(Double.valueOf(d2))
```

(Unlike the == operator, this method considers `NaN` equal to itself, and 0.0d unequal to -0.0d.)

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

See Also:

`Double.equals(Object)`

equals

```
public static boolean equals(float[] a,
                             float[] a2)
```

Returns `true` if the two specified arrays of floats are *equal* to one another. Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`. Two floats `f1` and `f2` are considered equal if:

```
    Float.valueOf(f1).equals(Float.valueOf(f2))
```

(Unlike the == operator, this method considers `NaN` equal to itself, and 0.0f unequal to -0.0f.)

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

See Also:

`Float.equals(Object)`

equals

```
public static boolean equals(float[] a,
                             int aFromIndex,
                             int aToIndex,
                             float[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of floats, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Two floats `f1` and `f2` are considered equal if:

```
    Float.valueOf(f1).equals(Float.valueOf(f2))
```

(Unlike the == operator, this method considers `NaN` equal to itself, and 0.0f unequal to -0.0f.)

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

See Also:

`Float.equals(Object)`

equals

```
public static boolean equals(Object[] a,
                             Object[] a2)
```

Returns `true` if the two specified arrays of Objects are *equal* to one another. The two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. Two objects `e1` and `e2` are considered *equal* if `Objects.equals(e1, e2)`. In other words, the two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

equals

```
public static boolean equals(Object[] a,
                             int aFromIndex,
                             int aToIndex,
                             Object[] b,
                             int bFromIndex,
                             int bToIndex)
```

Returns true if the two specified arrays of Objects, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Two objects `e1` and `e2` are considered *equal* if `Objects.equals(e1, e2)`.

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

equals

```
public static <T> boolean equals(T[] a,
                                 T[] a2,
                                 Comparator<? super T> cmp)
```

Returns `true` if the two specified arrays of Objects are *equal* to one another.

Two arrays are considered equal if both arrays contain the same number of elements, and all corresponding pairs of elements in the two arrays are equal. In other words, the two arrays are equal if they contain the same elements in the same order. Also, two array references are considered equal if both are `null`.

Two objects `e1` and `e2` are considered *equal* if, given the specified comparator, `cmp.compare(e1, e2) == 0`.

Type Parameters:

`T` - the type of array elements

Parameters:

`a` - one array to be tested for equality

`a2` - the other array to be tested for equality

`cmp` - the comparator to compare array elements

Returns:

`true` if the two arrays are equal

Throws:

`NullPointerException` - if the comparator is `null`

Since:

9

equals

```
public static <T> boolean equals(T[] a,
                                 int aFromIndex,
                                 int aToIndex,
                                 T[] b,
                                 int bFromIndex,
                                 int bToIndex,
                                 Comparator<? super T> cmp)
```

Returns true if the two specified arrays of Objects, over the specified ranges, are *equal* to one another.

Two arrays are considered equal if the number of elements covered by each range is the same, and all corresponding pairs of elements over the specified ranges in the two arrays are equal. In other words, two arrays are equal if they contain, over the specified ranges, the same elements in the same order.

Two objects `e1` and `e2` are considered *equal* if, given the specified comparator, `cmp.compare(e1, e2) == 0`.

Type Parameters:

`T` - the type of array elements

Parameters:

`a` - the first array to be tested for equality

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for equality

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

`cmp` - the comparator to compare array elements

Returns:

`true` if the two arrays, over the specified ranges, are equal

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

NullPointerException - if either array or the comparator is null

Since:

9

fill

```
public static void fill(long[] a,
                        long val)
```

Assigns the specified long value to each element of the specified array of longs.

Parameters:

a - the array to be filled

val - the value to be stored in all elements of the array

fill

```
public static void fill(long[] a,
                        int fromIndex,
                        int toIndex,
                        long val)
```

Assigns the specified long value to each element of the specified range of the specified array of longs. The range to be filled extends from index fromIndex, inclusive, to index toIndex, exclusive. (If fromIndex==toIndex, the range to be filled is empty.)

Parameters:

a - the array to be filled

fromIndex - the index of the first element (inclusive) to be filled with the specified value

toIndex - the index of the last element (exclusive) to be filled with the specified value

val - the value to be stored in all elements of the array

Throws:

IllegalArgumentException - if fromIndex > toIndex

ArrayIndexOutOfBoundsException - if fromIndex < 0 or toIndex > a.length

fill

```
public static void fill(int[] a,
                        int val)
```

Assigns the specified int value to each element of the specified array of ints.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

fill

```
public static void fill(int[] a,
                        int fromIndex,
                        int toIndex,
                        int val)
```

Assigns the specified int value to each element of the specified range of the specified array of ints. The range to be filled extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

fill

```
public static void fill(short[] a,
                        short val)
```

Assigns the specified short value to each element of the specified array of shorts.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

fill

```
public static void fill(short[] a,
                        int fromIndex,
                        int toIndex,
                        short val)
```

Assigns the specified short value to each element of the specified range of the specified array of shorts. The range to be filled extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

fill

```
public static void fill(char[] a,
                        char val)
```

Assigns the specified char value to each element of the specified array of chars.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

fill

```
public static void fill(char[] a,
                        int fromIndex,
                        int toIndex,
                        char val)
```

Assigns the specified char value to each element of the specified range of the specified array of chars. The range to be filled extends from index `fromIndex`,

inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

fill

```
public static void fill(byte[] a,
                        byte val)
```

Assigns the specified byte value to each element of the specified array of bytes.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

fill

```
public static void fill(byte[] a,
                        int fromIndex,
                        int toIndex,
                        byte val)
```

Assigns the specified byte value to each element of the specified range of the specified array of bytes. The range to be filled extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

fill

```
public static void fill(boolean[] a,
                        boolean val)
```

Assigns the specified boolean value to each element of the specified array of booleans.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

fill

```
public static void fill(boolean[] a,
                        int fromIndex,
                        int toIndex,
                        boolean val)
```

Assigns the specified boolean value to each element of the specified range of the specified array of booleans. The range to be filled extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

fill

```
public static void fill(double[] a,
                        double val)
```

Assigns the specified double value to each element of the specified array of doubles.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

fill

```
public static void fill(double[] a,
                        int fromIndex,
                        int toIndex,
                        double val)
```

Assigns the specified double value to each element of the specified range of the specified array of doubles. The range to be filled extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

fill

```
public static void fill(float[] a,
                        float val)
```

Assigns the specified float value to each element of the specified array of floats.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

fill

```
public static void fill(float[] a,
                        int fromIndex,
                        int toIndex,
                        float val)
```

Assigns the specified float value to each element of the specified range of the specified array of floats. The range to be filled extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

fill

```
public static void fill(Object[] a,
                        Object val)
```

Assigns the specified Object reference to each element of the specified array of Objects.

Parameters:

`a` - the array to be filled

`val` - the value to be stored in all elements of the array

Throws:

`ArrayStoreException` - if the specified value is not of a runtime type that can be stored in the specified array

fill

```
public static void fill(Object[] a,
                        int fromIndex,
                        int toIndex,
                        Object val)
```

Assigns the specified Object reference to each element of the specified range of the specified array of Objects. The range to be filled extends from index `fromIndex`, inclusive, to index `toIndex`, exclusive. (If `fromIndex==toIndex`, the range to be filled is empty.)

Parameters:

`a` - the array to be filled

`fromIndex` - the index of the first element (inclusive) to be filled with the specified value

`toIndex` - the index of the last element (exclusive) to be filled with the specified value

`val` - the value to be stored in all elements of the array

Throws:

`IllegalArgumentException` - if `fromIndex > toIndex`

`ArrayIndexOutOfBoundsException` - if `fromIndex < 0` or `toIndex > a.length`

`ArrayStoreException` - if the specified value is not of a runtime type that can be stored in the specified array

copyOf

```
public static <T> T[] copyOf(T[] original,
                             int newLength)
```

Copies the specified array, truncating or padding with nulls (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain `null`. Such indices will exist if and only if the specified length is greater than that of the original array. The resulting array is of exactly the same class as the original array.

Type Parameters:

`T` - the class of the objects in the array

Parameters:

`original` - the array to be copied

newLength - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with nulls to obtain the specified length

Throws:

NegativeArraySizeException - if newLength is negative

NullPointerException - if original is null

Since:

1.6

copyOf

```
public static <T,U> T[] copyOf(U[] original,
                               int newLength,
                               Class<? extends T[]> newType)
```

Copies the specified array, truncating or padding with nulls (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain null. Such indices will exist if and only if the specified length is greater than that of the original array. The resulting array is of the class newType.

Type Parameters:

T - the class of the objects in the returned array

U - the class of the objects in the original array

Parameters:

original - the array to be copied

newLength - the length of the copy to be returned

newType - the class of the copy to be returned

Returns:

a copy of the original array, truncated or padded with nulls to obtain the specified length

Throws:

NegativeArraySizeException - if newLength is negative

NullPointerException - if original is null

ArrayStoreException - if an element copied from original is not of a runtime type that can be stored in an array of class newType

Since:

1.6

copyOf

```
public static byte[] copyOf(byte[] original,
                            int newLength)
```

Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain `(byte)0`. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

`original` - the array to be copied

`newLength` - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with zeros to obtain the specified length

Throws:

`NegativeArraySizeException` - if `newLength` is negative

`NullPointerException` - if `original` is null

Since:

1.6

copyOf

```
public static short[] copyOf(short[] original,
                             int newLength)
```

Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain `(short)0`. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

`original` - the array to be copied

`newLength` - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with zeros to obtain the specified length

Throws:

`NegativeArraySizeException` - if `newLength` is negative

`NullPointerException` - if `original` is null

Since:

1.6

copyOf

```
public static int[] copyOf(int[] original,
                           int newLength)
```

Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain `0`. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

`original` - the array to be copied

`newLength` - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with zeros to obtain the specified length

Throws:

`NegativeArraySizeException` - if `newLength` is negative

`NullPointerException` - if `original` is null

Since:

1.6

copyOf

```
public static long[] copyOf(long[] original,
                            int newLength)
```

Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will

contain `0L`. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

`original` - the array to be copied

`newLength` - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with zeros to obtain the specified length

Throws:

`NegativeArraySizeException` - if `newLength` is negative

`NullPointerException` - if `original` is null

Since:

1.6

copyOf

```
public static char[] copyOf(char[] original,
                            int newLength)
```

Copies the specified array, truncating or padding with null characters (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain `'\u0000'`. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

`original` - the array to be copied

`newLength` - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with null characters to obtain the specified length

Throws:

`NegativeArraySizeException` - if `newLength` is negative

`NullPointerException` - if `original` is null

Since:

1.6

copyOf

```
public static float[] copyOf(float[] original,
                             int newLength)
```

Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain `0f`. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

`original` - the array to be copied

`newLength` - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with zeros to obtain the specified length

Throws:

`NegativeArraySizeException` - if `newLength` is negative

`NullPointerException` - if `original` is null

Since:

1.6

copyOf

```
public static double[] copyOf(double[] original,
                              int newLength)
```

Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain `0d`. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

`original` - the array to be copied

`newLength` - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with zeros to obtain the specified length

Throws:

NegativeArraySizeException - if newLength is negative

NullPointerException - if original is null

Since:

1.6

copyOf

```
public static boolean[] copyOf(boolean[] original,
                               int newLength)
```

Copies the specified array, truncating or padding with false (if necessary) so the copy has the specified length. For all indices that are valid in both the original array and the copy, the two arrays will contain identical values. For any indices that are valid in the copy but not the original, the copy will contain false. Such indices will exist if and only if the specified length is greater than that of the original array.

Parameters:

original - the array to be copied

newLength - the length of the copy to be returned

Returns:

a copy of the original array, truncated or padded with false elements to obtain the specified length

Throws:

NegativeArraySizeException - if newLength is negative

NullPointerException - if original is null

Since:

1.6

copyOfRange

```
public static <T> T[] copyOfRange(T[] original,
                                  int from,
                                  int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (from) must lie between zero and original.length, inclusive. The value at original[from] is placed into the initial element of the copy (unless from == original.length or from == to). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (to), which must be greater than or equal to from, may be greater than original.length, in which case null is placed in all elements of the copy whose index is greater than or equal

to `original.length - from`. The length of the returned array will be `to - from`.

The resulting array is of exactly the same class as the original array.

Type Parameters:

`T` - the class of the objects in the array

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with nulls to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

`NullPointerException` - if `original` is null

Since:

1.6

copyOfRange

```
public static <T,U> T[] copyOfRange(U[] original,
                                    int from,
                                    int to,
                                    Class<? extends T[]> newType)
```

Copies the specified range of the specified array into a new array. The initial index of the range (`from`) must lie between zero and `original.length`, inclusive. The value at `original[from]` is placed into the initial element of the copy (unless `from == original.length` or `from == to`). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (`to`), which must be greater than or equal to `from`, may be greater than `original.length`, in which case `null` is placed in all elements of the copy whose index is greater than or equal to `original.length - from`. The length of the returned array will be `to - from`. The resulting array is of the class `newType`.

Type Parameters:

`T` - the class of the objects in the returned array

`U` - the class of the objects in the original array

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

`newType` - the class of the copy to be returned

Returns:

a new array containing the specified range from the original array, truncated or padded with nulls to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

`NullPointerException` - if `original` is null

`ArrayStoreException` - if an element copied from `original` is not of a runtime type that can be stored in an array of class `newType`.

Since:

1.6

copyOfRange

```
public static byte[] copyOfRange(byte[] original,
                                 int from,
                                 int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (`from`) must lie between zero and `original.length`, inclusive. The value at `original[from]` is placed into the initial element of the copy (unless `from == original.length` or `from == to`). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (`to`), which must be greater than or equal to `from`, may be greater than `original.length`, in which case `(byte)0` is placed in all elements of the copy whose index is greater than or equal to `original.length - from`. The length of the returned array will be `to - from`.

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with zeros to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

`NullPointerException` - if `original` is null

Since:

1.6

copyOfRange

```
public static short[] copyOfRange(short[] original,
                                  int from,
                                  int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (`from`) must lie between zero and `original.length`, inclusive. The value at `original[from]` is placed into the initial element of the copy (unless `from == original.length` or `from == to`). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (`to`), which must be greater than or equal to `from`, may be greater than `original.length`, in which case `(short)0` is placed in all elements of the copy whose index is greater than or equal to `original.length - from`. The length of the returned array will be `to - from`.

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with zeros to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

NullPointerException - if original is null

Since:

1.6

copyOfRange

```
public static int[] copyOfRange(int[] original,
                                int from,
                                int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (from) must lie between zero and original.length, inclusive. The value at original[from] is placed into the initial element of the copy (unless from == original.length or from == to). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (to), which must be greater than or equal to from, may be greater than original.length, in which case 0 is placed in all elements of the copy whose index is greater than or equal to original.length - from. The length of the returned array will be to - from.

Parameters:

original - the array from which a range is to be copied

from - the initial index of the range to be copied, inclusive

to - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with zeros to obtain the required length

Throws:

ArrayIndexOutOfBoundsException - if from < 0 or from > original.length

IllegalArgumentException - if from > to

NullPointerException - if original is null

Since:

1.6

copyOfRange

```
public static long[] copyOfRange(long[] original,
                                 int from,
                                 int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (from) must lie between zero and original.length,

inclusive. The value at `original[from]` is placed into the initial element of the copy (unless `from == original.length` or `from == to`). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (`to`), which must be greater than or equal to `from`, may be greater than `original.length`, in which case `0L` is placed in all elements of the copy whose index is greater than or equal to `original.length - from`. The length of the returned array will be `to - from`.

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with zeros to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

`NullPointerException` - if `original` is null

Since:

1.6

copyOfRange

```
public static char[] copyOfRange(char[] original,
                                 int from,
                                 int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (`from`) must lie between zero and `original.length`, inclusive. The value at `original[from]` is placed into the initial element of the copy (unless `from == original.length` or `from == to`). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (`to`), which must be greater than or equal to `from`, may be greater than `original.length`, in which case `'\u0000'` is placed in all elements of the copy whose index is greater than or equal to `original.length - from`. The length of the returned array will be `to - from`.

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with null characters to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

`NullPointerException` - if `original` is null

Since:

1.6

copyOfRange

```
public static float[] copyOfRange(float[] original,
                                  int from,
                                  int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (`from`) must lie between zero and `original.length`, inclusive. The value at `original[from]` is placed into the initial element of the copy (unless `from == original.length` or `from == to`). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (`to`), which must be greater than or equal to `from`, may be greater than `original.length`, in which case `0f` is placed in all elements of the copy whose index is greater than or equal to `original.length - from`. The length of the returned array will be `to - from`.

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with zeros to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

NullPointerException - if original is null

Since:

1.6

copyOfRange

```
public static double[] copyOfRange(double[] original,
                                   int from,
                                   int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (from) must lie between zero and original.length, inclusive. The value at original[from] is placed into the initial element of the copy (unless from == original.length or from == to). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (to), which must be greater than or equal to from, may be greater than original.length, in which case 0d is placed in all elements of the copy whose index is greater than or equal to original.length - from. The length of the returned array will be to - from.

Parameters:

original - the array from which a range is to be copied

from - the initial index of the range to be copied, inclusive

to - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with zeros to obtain the required length

Throws:

ArrayIndexOutOfBoundsException - if from < 0 or from > original.length

IllegalArgumentException - if from > to

NullPointerException - if original is null

Since:

1.6

copyOfRange

```
public static boolean[] copyOfRange(boolean[] original,
                                    int from,
                                    int to)
```

Copies the specified range of the specified array into a new array. The initial index of the range (from) must lie between zero and original.length,

inclusive. The value at `original[from]` is placed into the initial element of the copy (unless `from == original.length` or `from == to`). Values from subsequent elements in the original array are placed into subsequent elements in the copy. The final index of the range (`to`), which must be greater than or equal to `from`, may be greater than `original.length`, in which case `false` is placed in all elements of the copy whose index is greater than or equal to `original.length - from`. The length of the returned array will be `to - from`.

Parameters:

`original` - the array from which a range is to be copied

`from` - the initial index of the range to be copied, inclusive

`to` - the final index of the range to be copied, exclusive. (This index may lie outside the array.)

Returns:

a new array containing the specified range from the original array, truncated or padded with false elements to obtain the required length

Throws:

`ArrayIndexOutOfBoundsException` - if `from < 0` or `from > original.length`

`IllegalArgumentException` - if `from > to`

`NullPointerException` - if `original` is null

Since:

1.6

asList

```
@SafeVarargs
public static <T> List<T> asList(T... a)
```

Returns a fixed-size list backed by the specified array. Changes made to the array will be visible in the returned list, and changes made to the list will be visible in the array. The returned list is `Serializable` and implements `RandomAccess`.

The returned list implements the optional `Collection` methods, except those that would change the size of the returned list. Those methods leave the list unchanged and throw `UnsupportedOperationException`.

If the specified array's actual component type differs from the type parameter T, this can result in operations on the returned list throwing an `ArrayStoreException`.

API Note:

This method acts as bridge between array-based and collection-based APIs, in combination with `Collection.toArray()`.

This method provides a way to wrap an existing array:

```
    Integer[] numbers = ...
    ...
    List<Integer> values = Arrays.asList(numbers);
```

This method also provides a convenient way to create a fixed-size list initialized to contain several elements:

```
    List<String> stooges = Arrays.asList("Larry", "Moe", "Curly");
```

The list returned by this method is modifiable. To create an unmodifiable list, use `Collections.unmodifiableList` or Unmodifiable Lists.

Type Parameters:

`T` - the class of the objects in the array

Parameters:

`a` - the array by which the list will be backed

Returns:

a list view of the specified array

Throws:

`NullPointerException` - if the specified array is `null`

hashCode

`public static int hashCode(long[] a)`

Returns a hash code based on the contents of the specified array. For any two `long` arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Long` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(int[] a)
```

Returns a hash code based on the contents of the specified array. For any two non-null `int` arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Integer` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(short[] a)
```

Returns a hash code based on the contents of the specified array. For any two `short` arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Short` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(char[] a)
```

Returns a hash code based on the contents of the specified array. For any two `char` arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Character` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(byte[] a)
```

Returns a hash code based on the contents of the specified array. For any two `byte` arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Byte` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(boolean[] a)
```

Returns a hash code based on the contents of the specified array. For any two `boolean` arrays `a` and `b` such that `Arrays.equals(a, b)`,
it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Boolean` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(float[] a)
```

Returns a hash code based on the contents of the specified array. For any two `float` arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Float` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(double[] a)
```

Returns a hash code based on the contents of the specified array. For any two `double` arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is the same value that would be obtained by invoking the `hashCode` method on a `List` containing a sequence of `Double` instances representing the elements of `a` in the same order. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose hash value to compute

Returns:

a content-based hash code for `a`

Since:

1.5

hashCode

```
public static int hashCode(Object[] a)
```

Returns a hash code based on the contents of the specified array. If the array contains other arrays as elements, the hash code is based on their identities rather than their contents. It is therefore acceptable to invoke this method on an array that contains itself as an element, either directly or indirectly through one or more levels of arrays.

For any two arrays `a` and `b` such that `Arrays.equals(a, b)`, it is also the case that `Arrays.hashCode(a) == Arrays.hashCode(b)`.

The value returned by this method is equal to the value that would be returned by `Arrays.asList(a).hashCode()`, unless `a` is `null`, in which case `0` is returned.

Parameters:

`a` - the array whose content-based hash code to compute

Returns:

a content-based hash code for `a`

Since:

1.5

See Also:

`deepHashCode(Object[])`

deepHashCode

```
public static int deepHashCode(Object[] a)
```

Returns a hash code based on the "deep contents" of the specified array. If the array contains other arrays as elements, the hash code is based on their

contents and so on, ad infinitum. It is therefore unacceptable to invoke this method on an array that contains itself as an element, either directly or indirectly through one or more levels of arrays. The behavior of such an invocation is undefined.

For any two arrays `a` and `b` such that `Arrays.deepEquals(a, b)`, it is also the case that `Arrays.deepHashCode(a) == Arrays.deepHashCode(b)`.

The computation of the value returned by this method is similar to that of the value returned by `List.hashCode()` on a list containing the same elements as `a` in the same order, with one difference: If an element `e` of `a` is itself an array, its hash code is computed not by calling `e.hashCode()`, but as by calling the appropriate overloading of `Arrays.hashCode(e)` if `e` is an array of a primitive type, or as by calling `Arrays.deepHashCode(e)` recursively if `e` is an array of a reference type. If `a` is `null`, this method returns 0.

Parameters:

`a` - the array whose deep-content-based hash code to compute

Returns:

a deep-content-based hash code for `a`

Since:

1.5

See Also:

`hashCode(Object[])`

deepEquals

```
public static boolean deepEquals(Object[] a1,
                                 Object[] a2)
```

Returns `true` if the two specified arrays are *deeply equal* to one another. Unlike the `equals(Object[],Object[])` method, this method is appropriate for use with nested arrays of arbitrary depth.

Two array references are considered deeply equal if both are `null`, or if they refer to arrays that contain the same number of elements and all corresponding pairs of elements in the two arrays are deeply equal.

Two possibly `null` elements `e1` and `e2` are deeply equal if any of the following conditions hold:

- `e1` and `e2` are both arrays of object reference types, and `Arrays.deepEquals(e1, e2) would return true`
- `e1` and `e2` are arrays of the same primitive type, and the appropriate overloading of `Arrays.equals(e1, e2)` would return true.
- `e1 == e2`

- `e1.equals(e2)` would return true.

Note that this definition permits `null` elements at any depth.

If either of the specified arrays contain themselves as elements either directly or indirectly through one or more levels of arrays, the behavior of this method is undefined.

Parameters:

`a1` - one array to be tested for equality

`a2` - the other array to be tested for equality

Returns:

`true` if the two arrays are equal

Since:

1.5

See Also:

`equals(Object[],Object[])`,
`Objects.deepEquals(Object, Object)`

toString

```
public static String toString(long[] a)
```

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `", "` (a comma followed by a space). Elements are converted to strings as by `String.valueOf(long)`. Returns `"null"` if `a` is `null`.

Parameters:

`a` - the array whose string representation to return

Returns:

a string representation of `a`

Since:

1.5

toString

```
public static String toString(int[] a)
```

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `",`

" (a comma followed by a space). Elements are converted to strings as by `String.valueOf(int)`. Returns `"null"` if `a` is `null`.

Parameters:

`a` - the array whose string representation to return

Returns:

a string representation of `a`

Since:

1.5

toString

```
public static String toString(short[] a)
```

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `", "` (a comma followed by a space). Elements are converted to strings as by `String.valueOf(short)`. Returns `"null"` if `a` is `null`.

Parameters:

`a` - the array whose string representation to return

Returns:

a string representation of `a`

Since:

1.5

toString

```
public static String toString(char[] a)
```

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `", "` (a comma followed by a space). Elements are converted to strings as by `String.valueOf(char)`. Returns `"null"` if `a` is `null`.

Parameters:

`a` - the array whose string representation to return

Returns:

a string representation of `a`

Since:

1.5

toString

`public static String toString(byte[] a)`

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `", "` (a comma followed by a space). Elements are converted to strings as by `String.valueOf(byte)`. Returns `"null"` if `a` is `null`.

Parameters:

`a` - the array whose string representation to return

Returns:

a string representation of `a`

Since:

1.5

toString

`public static String toString(boolean[] a)`

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `", "` (a comma followed by a space). Elements are converted to strings as by `String.valueOf(boolean)`. Returns `"null"` if `a` is `null`.

Parameters:

`a` - the array whose string representation to return

Returns:

a string representation of `a`

Since:

1.5

toString

`public static String toString(float[] a)`

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `", "` (a comma followed by a space). Elements are converted to strings as by `String.valueOf(float)`. Returns `"null"` if `a` is `null`.

Parameters:

a - the array whose string representation to return

Returns:

a string representation of a

Since:

1.5

toString

```
public static String toString(double[] a)
```

Returns a string representation of the contents of the specified array. The string representation consists of a list of the array's elements, enclosed in square brackets ("[]"). Adjacent elements are separated by the characters ", " (a comma followed by a space). Elements are converted to strings as by String.valueOf(double). Returns "null" if a is null.

Parameters:

a - the array whose string representation to return

Returns:

a string representation of a

Since:

1.5

toString

```
public static String toString(Object[] a)
```

Returns a string representation of the contents of the specified array. If the array contains other arrays as elements, they are converted to strings by the Object.toString() method inherited from Object, which describes their *identities* rather than their contents.

The value returned by this method is equal to the value that would be returned by Arrays.asList(a).toString(), unless a is null, in which case "null" is returned.

Parameters:

a - the array whose string representation to return

Returns:

a string representation of a

Since:

1.5

See Also:

`deepToString(Object[])`

deepToString

```
public static String deepToString(Object[] a)
```

Returns a string representation of the "deep contents" of the specified array. If the array contains other arrays as elements, the string representation contains their contents and so on. This method is designed for converting multidimensional arrays to strings.

The string representation consists of a list of the array's elements, enclosed in square brackets (`"[]"`). Adjacent elements are separated by the characters `", "` (a comma followed by a space). Elements are converted to strings as by `String.valueOf(Object)`, unless they are themselves arrays.

If an element `e` is an array of a primitive type, it is converted to a string as by invoking the appropriate overloading of `Arrays.toString(e)`. If an element `e` is an array of a reference type, it is converted to a string as by invoking this method recursively.

To avoid infinite recursion, if the specified array contains itself as an element, or contains an indirect reference to itself through one or more levels of arrays, the self-reference is converted to the string `"[...]"`. For example, an array containing only a reference to itself would be rendered as `"[[...]]"`.

This method returns `"null"` if the specified array is `null`.

Parameters:

`a` - the array whose string representation to return

Returns:

a string representation of `a`

Since:

1.5

See Also:

`toString(Object[])`

setAll

```
public static <T> void setAll(T[] array,
                              IntFunction<? extends T> generator)
```

Set all elements of the specified array, using the provided generator function to compute each element.

If the generator function throws an exception, it is relayed to the caller and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .forEach(i -> array[i] = generator.apply(i));
```

Type Parameters:

T - type of elements of the array

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

`NullPointerException` - if the generator is null

Since:

1.8

parallelSetAll

```
public static <T> void parallelSetAll
(T[] array,
 IntFunction<? extends T> generator)
```

Set all elements of the specified array, in parallel, using the provided generator function to compute each element.

If the generator function throws an exception, an unchecked exception is thrown from `parallelSetAll` and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, in parallel, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .parallel()
         .forEach(i -> array[i] = generator.apply(i));
```

Type Parameters:

T - type of elements of the array

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

`NullPointerException` - if the generator is null

Since:

1.8

setAll

```
public static void setAll(int[] array,
                          IntUnaryOperator generator)
```

Set all elements of the specified array, using the provided generator function to compute each element.

If the generator function throws an exception, it is relayed to the caller and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .forEach(i -> array[i] = generator.applyAsInt(i));
```

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

`NullPointerException` - if the generator is null

Since:

1.8

parallelSetAll

```
public static void parallelSetAll(int[] array,
                                  IntUnaryOperator generator)
```

Set all elements of the specified array, in parallel, using the provided generator function to compute each element.

If the generator function throws an exception, an unchecked exception is thrown from `parallelSetAll` and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, in parallel, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .parallel()
         .forEach(i -> array[i] = generator.applyAsInt(i));
```

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

`NullPointerException` - if the generator is null

Since:

1.8

setAll

```
public static void setAll(long[] array,
                          IntToLongFunction generator)
```

Set all elements of the specified array, using the provided generator function to compute each element.

If the generator function throws an exception, it is relayed to the caller and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .forEach(i -> array[i] = generator.applyAsLong(i));
```

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

NullPointerException - if the generator is null

Since:

1.8

parallelSetAll

```
public static void parallelSetAll(long[] array,
                                  IntToLongFunction generator)
```

Set all elements of the specified array, in parallel, using the provided generator function to compute each element.

If the generator function throws an exception, an unchecked exception is thrown from `parallelSetAll` and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, in parallel, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .parallel()
         .forEach(i -> array[i] = generator.applyAsLong(i));
```

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

NullPointerException - if the generator is null

Since:

1.8

setAll

```
public static void setAll(double[] array,
                          IntToDoubleFunction generator)
```

Set all elements of the specified array, using the provided generator function to compute each element.

If the generator function throws an exception, it is relayed to the caller and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .forEach(i -> array[i] = generator.applyAsDouble(i));
```

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

`NullPointerException` - if the generator is null

Since:

1.8

parallelSetAll

```
public static void parallelSetAll(double[] array,
                                  IntToDoubleFunction generator)
```

Set all elements of the specified array, in parallel, using the provided generator function to compute each element.

If the generator function throws an exception, an unchecked exception is thrown from `parallelSetAll` and the array is left in an indeterminate state.

API Note:

Setting a subrange of an array, in parallel, using a generator function to compute each element, can be written as follows:

```
IntStream.range(startInclusive, endExclusive)
         .parallel()
         .forEach(i -> array[i] = generator.applyAsDouble(i));
```

Parameters:

`array` - array to be initialized

`generator` - a function accepting an index and producing the desired value for that position

Throws:

`NullPointerException` - if the generator is null

Since:

1.8

spliterator

```
public static <T> Spliterator<T> spliterator(T[] array)
```

Returns a `Spliterator` covering all of the specified array.

The spliterator reports `Spliterator.SIZED`, `Spliterator.SUBSIZED`, `Spliterator.ORDERED`, and `Spliterator.IMMUTABLE`.

Type Parameters:

T - type of elements

Parameters:

`array` - the array, assumed to be unmodified during use

Returns:

a spliterator for the array elements

Since:

1.8

spliterator

```
public static <T> Spliterator<T> spliterator(T[] array,
                                             int startInclusive,
                                             int endExclusive)
```

Returns a `Spliterator` covering the specified range of the specified array.

The spliterator reports `Spliterator.SIZED`, `Spliterator.SUBSIZED`, `Spliterator.ORDERED`, and `Spliterator.IMMUTABLE`.

Type Parameters:

T - type of elements

Parameters:

`array` - the array, assumed to be unmodified during use

`startInclusive` - the first index to cover, inclusive

`endExclusive` - index immediately past the last index to cover

Returns:

a spliterator for the array elements

Throws:

`ArrayIndexOutOfBoundsException` - if `startInclusive` is negative, `endExclusive` is less than `startInclusive`, or `endExclusive` is greater than the array size

Since:

1.8

spliterator

```
public static Spliterator.OfInt spliterator(int[] array)
```

Returns a `Spliterator.OfInt` covering all of the specified array.

The spliterator reports `Spliterator.SIZED`, `Spliterator.SUBSIZED`, `Spliterator.ORDERED`, and `Spliterator.IMMUTABLE`.

Parameters:

`array` - the array, assumed to be unmodified during use

Returns:

a spliterator for the array elements

Since:

1.8

spliterator

```
public static Spliterator.OfInt spliterator(int[] array,
                                            int startInclusive,
                                            int endExclusive)
```

Returns a `Spliterator.OfInt` covering the specified range of the specified array.

The spliterator reports `Spliterator.SIZED`, `Spliterator.SUBSIZED`, `Spliterator.ORDERED`, and `Spliterator.IMMUTABLE`.

Parameters:

`array` - the array, assumed to be unmodified during use

`startInclusive` - the first index to cover, inclusive

`endExclusive` - index immediately past the last index to cover

Returns:

a spliterator for the array elements

Throws:

`ArrayIndexOutOfBoundsException` - if `startInclusive` is negative, `endExclusive` is less than `startInclusive`, or `endExclusive` is greater than the array size

Since:

1.8

spliterator

```
public static Spliterator.OfLong spliterator(long[] array)
```

Returns a Spliterator.OfLong covering all of the specified array.

The spliterator reports Spliterator.SIZED, Spliterator.SUBSIZED, Spliterator.ORDERED, and Spliterator.IMMUTABLE.

Parameters:

array - the array, assumed to be unmodified during use

Returns:

the spliterator for the array elements

Since:

1.8

spliterator

```
public static Spliterator.OfLong spliterator(long[] array,
                                             int startInclusive,
                                             int endExclusive)
```

Returns a Spliterator.OfLong covering the specified range of the specified array.

The spliterator reports Spliterator.SIZED, Spliterator.SUBSIZED, Spliterator.ORDERED, and Spliterator.IMMUTABLE.

Parameters:

array - the array, assumed to be unmodified during use

startInclusive - the first index to cover, inclusive

endExclusive - index immediately past the last index to cover

Returns:

a spliterator for the array elements

Throws:

ArrayIndexOutOfBoundsException - if startInclusive is negative, endExclusive is less than startInclusive, or endExclusive is greater than the array size

Since:

1.8

spliterator

```
public static Spliterator.OfDouble spliterator(double[] array)
```

Returns a Spliterator.OfDouble covering all of the specified array.

The spliterator reports Spliterator.SIZED, Spliterator.SUBSIZED, Spliterator.ORDERED, and Spliterator.IMMUTABLE.

Parameters:

array - the array, assumed to be unmodified during use

Returns:

a spliterator for the array elements

Since:

1.8

spliterator

```
public static Spliterator.OfDouble spliterator(double[] array,
                                               int startInclusive,
                                               int endExclusive)
```

Returns a Spliterator.OfDouble covering the specified range of the specified array.

The spliterator reports Spliterator.SIZED, Spliterator.SUBSIZED, Spliterator.ORDERED, and Spliterator.IMMUTABLE.

Parameters:

array - the array, assumed to be unmodified during use

startInclusive - the first index to cover, inclusive

endExclusive - index immediately past the last index to cover

Returns:

a spliterator for the array elements

Throws:

ArrayIndexOutOfBoundsException - if startInclusive is negative, endExclusive is less than startInclusive, or endExclusive is greater than the array size

Since:

1.8

stream

```
public static <T> Stream<T> stream(T[] array)
```

Returns a sequential Stream with the specified array as its source.

Type Parameters:

T - The type of the array elements

Parameters:

array - The array, assumed to be unmodified during use

Returns:

a Stream for the array

Since:

1.8

stream

```
public static <T> Stream<T> stream(T[] array,
                                   int startInclusive,
                                   int endExclusive)
```

Returns a sequential Stream with the specified range of the specified array as its source.

Type Parameters:

T - the type of the array elements

Parameters:

array - the array, assumed to be unmodified during use

startInclusive - the first index to cover, inclusive

endExclusive - index immediately past the last index to cover

Returns:

a Stream for the array range

Throws:

ArrayIndexOutOfBoundsException - if startInclusive is negative, endExclusive is less than startInclusive, or endExclusive is greater than the array size

Since:

1.8

stream

```
public static IntStream stream(int[] array)
```

Returns a sequential IntStream with the specified array as its source.

Parameters:

array - the array, assumed to be unmodified during use

Returns:

an IntStream for the array

Since:

1.8

stream

```
public static IntStream stream(int[] array,
                               int startInclusive,
                               int endExclusive)
```

Returns a sequential `IntStream` with the specified range of the specified array as its source.

Parameters:

`array` - the array, assumed to be unmodified during use

`startInclusive` - the first index to cover, inclusive

`endExclusive` - index immediately past the last index to cover

Returns:

an `IntStream` for the array range

Throws:

`ArrayIndexOutOfBoundsException` - if `startInclusive` is negative, `endExclusive` is less than `startInclusive`, or `endExclusive` is greater than the array size

Since:

1.8

stream

```
public static LongStream stream(long[] array)
```

Returns a sequential `LongStream` with the specified array as its source.

Parameters:

`array` - the array, assumed to be unmodified during use

Returns:

a `LongStream` for the array

Since:

1.8

stream

```
public static LongStream stream(long[] array,
                                int startInclusive,
                                int endExclusive)
```

Returns a sequential `LongStream` with the specified range of the specified array as its source.

Parameters:

`array` - the array, assumed to be unmodified during use

`startInclusive` - the first index to cover, inclusive

`endExclusive` - index immediately past the last index to cover

Returns:

a `LongStream` for the array range

Throws:

`ArrayIndexOutOfBoundsException` - if `startInclusive` is negative, `endExclusive` is less than `startInclusive`, or `endExclusive` is greater than the array size

Since:

1.8

stream

```
public static DoubleStream stream(double[] array)
```

Returns a sequential `DoubleStream` with the specified array as its source.

Parameters:

`array` - the array, assumed to be unmodified during use

Returns:

a `DoubleStream` for the array

Since:

1.8

stream

```
public static DoubleStream stream(double[] array,
                                  int startInclusive,
                                  int endExclusive)
```

Returns a sequential `DoubleStream` with the specified range of the specified array as its source.

Parameters:

`array` - the array, assumed to be unmodified during use

`startInclusive` - the first index to cover, inclusive

`endExclusive` - index immediately past the last index to cover

Returns:

a `DoubleStream` for the array range

Throws:

`ArrayIndexOutOfBoundsException` - if `startInclusive` is negative, `endExclusive` is less than `startInclusive`, or `endExclusive` is greater than the array size

Since:

1.8

compare

```
public static int compare(boolean[] a,
                          boolean[] b)
```

Compares two `boolean` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Boolean.compare(boolean, boolean)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(boolean[], boolean[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Boolean.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static int compare(boolean[] a,
                          int aFromIndex,
                          int aToIndex,
                          boolean[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two `boolean` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Boolean.compare(boolean, boolean)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(boolean[], int, int, boolean[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Boolean.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compare

```
public static int compare(byte[] a,
                          byte[] b)
```

Compares two `byte` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Byte.compare(byte, byte)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(byte[], byte[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Byte.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static int compare(byte[] a,
                          int aFromIndex,
                          int aToIndex,
                          byte[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two `byte` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Byte.compare(byte, byte)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(byte[], int, int, byte[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Byte.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compareUnsigned

```
public static int compareUnsigned(byte[] a,
                                  byte[] b)
```

Compares two `byte` arrays lexicographically, numerically treating elements as unsigned.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Byte.compareUnsigned(byte, byte)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(byte[], byte[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Byte.compareUnsigned(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compareUnsigned

```
public static int compareUnsigned(byte[] a,
                                  int aFromIndex,
                                  int aToIndex,
                                  byte[] b,
                                  int bFromIndex,
                                  int bToIndex)
```

Compares two `byte` arrays lexicographically over the specified ranges, numerically treating elements as unsigned.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Byte.compareUnsigned(byte, byte)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(byte[], int, int, byte[], int, int)` for the definition of a common and proper prefix.)

API Note:

This method behaves as if:

```
int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                        b, bFromIndex, bToIndex);
if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
    return Byte.compareUnsigned(a[aFromIndex + i], b[bFromIndex + i]);
return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is null

Since:

9

compare

```
public static int compare(short[] a,
                          short[] b)
```

Compares two `short` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Short.compare(short, short)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(short[], short[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Short.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static int compare(short[] a,
                          int aFromIndex,
                          int aToIndex,
                          short[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two `short` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Short.compare(short, short)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(short[], int, int, short[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

```
        return Short.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compareUnsigned

```
public static int compareUnsigned(short[] a,
                                  short[] b)
```

Compares two `short` arrays lexicographically, numerically treating elements as unsigned.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Short.compareUnsigned(short, short)`, at an index within the respective

arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(short[], short[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

API Note:

This method behaves as if (for non-`null` array references):

```
int i = Arrays.mismatch(a, b);
if (i >= 0 && i < Math.min(a.length, b.length))
    return Short.compareUnsigned(a[i], b[i]);
return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compareUnsigned

```
public static int compareUnsigned(short[] a,
                                  int aFromIndex,
                                  int aToIndex,
                                  short[] b,
                                  int bFromIndex,
                                  int bToIndex)
```

Compares two `short` arrays lexicographically over the specified ranges, numerically treating elements as unsigned.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Short.compareUnsigned(short, short)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of

comparing the two range lengths. (See `mismatch(short[], int, int, short[], int, int)` for the definition of a common and proper prefix.)

API Note:

This method behaves as if:

```
int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                        b, bFromIndex, bToIndex);
if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
    return Short.compareUnsigned(a[aFromIndex + i], b[bFromIndex + i]);
return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is null

Since:

9

compare

```
public static int compare(char[] a,
                          char[] b)
```

Compares two `char` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Character.compare(char, char)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(char[], char[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Character.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static int compare(char[] a,
                          int aFromIndex,
                          int aToIndex,
                          char[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two `char` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Character.compare(char, char)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(char[], int, int, char[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Character.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compare

```
public static int compare(int[] a,
                          int[] b)
```

Compares two `int` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Integer.compare(int, int)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(int[], int[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Integer.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

a - the first array to compare

b - the second array to compare

Returns:

the value 0 if the first and second array are equal and contain the same elements in the same order; a value less than 0 if the first array is lexicographically less than the second array; and a value greater than 0 if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static int compare(int[] a,
                          int aFromIndex,
                          int aToIndex,
                          int[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two int arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by Integer.compare(int, int), at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See mismatch(int[], int, int, int[], int, int) for the definition of a common and proper prefix.)

The comparison is consistent with equals, more specifically the following holds for arrays a and b with specified ranges [aFromIndex, aToIndex) and [bFromIndex, bToIndex) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Integer.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compareUnsigned

```
public static int compareUnsigned(int[] a,
                                  int[] b)
```

Compares two `int` arrays lexicographically, numerically treating elements as unsigned.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Integer.compareUnsigned(int, int)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(int[], int[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

API Note:

This method behaves as if (for non-`null` array references):

```
int i = Arrays.mismatch(a, b);
if (i >= 0 && i < Math.min(a.length, b.length))
    return Integer.compareUnsigned(a[i], b[i]);
return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compareUnsigned

```
public static int compareUnsigned(int[] a,
                                  int aFromIndex,
                                  int aToIndex,
                                  int[] b,
                                  int bFromIndex,
                                  int bToIndex)
```

Compares two `int` arrays lexicographically over the specified ranges, numerically treating elements as unsigned.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Integer.compareUnsigned(int, int)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(int[], int, int, int[], int, int)` for the definition of a common and proper prefix.)

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Integer.compareUnsigned(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is null

Since:

9

compare

```
public static int compare(long[] a,
                          long[] b)
```

Compares two `long` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Long.compare(long, long)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(long[], long[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Long.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static int compare(long[] a,
                          int aFromIndex,
                          int aToIndex,
                          long[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two `long` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Long.compare(long, long)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(long[], int, int, long[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Long.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compareUnsigned

```
public static int compareUnsigned(long[] a,
                                  long[] b)
```

Compares two `long` arrays lexicographically, numerically treating elements as unsigned.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Long.compareUnsigned(long, long)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(long[], long[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
```

```
        return Long.compareUnsigned(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compareUnsigned

```
public static int compareUnsigned(long[] a,
                                  int aFromIndex,
                                  int aToIndex,
                                  long[] b,
                                  int bFromIndex,
                                  int bToIndex)
```

Compares two `long` arrays lexicographically over the specified ranges, numerically treating elements as unsigned.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Long.compareUnsigned(long, long)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(long[], int, int, long[], int, int)` for the definition of a common and proper prefix.)

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Long.compareUnsigned(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is null

Since:

9

compare

```
public static int compare(float[] a,
                          float[] b)
```

Compares two `float` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Float.compare(float, float)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(float[], float[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Float.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static int compare(float[] a,
                          int aFromIndex,
                          int aToIndex,
                          float[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two `float` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Float.compare(float, float)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(float[], int, int, float[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Float.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compare

```
public static int compare(double[] a,
                          double[] b)
```

Compares two `double` arrays lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Double.compare(double, double)`, at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(double[], double[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return Double.compare(a[i], b[i]);
    return a.length - b.length;
```

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

compare

```
public static int compare(double[] a,
                          int aFromIndex,
                          int aToIndex,
                          double[] b,
                          int bFromIndex,
                          int bToIndex)
```

Compares two `double` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements, as if by `Double.compare(double, double)`, at a relative index within the respective arrays that is the length of the prefix. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(double[], int, int, double[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if:

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return Double.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compare

```
public static <T extends Comparable<? super T>> int compare(T[] a,
                                                             T[] b)
```

Compares two `Object` arrays, within comparable elements, lexicographically.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing two elements of type `T` at an index `i` within the respective arrays that is the prefix length, as if by:

```
    Comparator.nullsFirst(Comparator.<T>naturalOrder()).
        compare(a[i], b[i])
```

Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(Object[], Object[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal. A `null` array element is considered lexicographically less than a non-`null` array element. Two `null` array elements are considered equal.

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b`:

```
    Arrays.equals(a, b) == (Arrays.compare(a, b) == 0)
```

API Note:

This method behaves as if (for non-`null` array references and elements):

```
    int i = Arrays.mismatch(a, b);
    if (i >= 0 && i < Math.min(a.length, b.length))
        return a[i].compareTo(b[i]);
    return a.length - b.length;
```

Type Parameters:

`T` - the type of comparable array elements

Parameters:

`a` - the first array to compare

`b` - the second array to compare

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Since:

9

compare

```
public static <T extends Comparable<? super T>> int compare
(T[] a,
 int aFromIndex,
 int aToIndex,
 T[] b,
 int bFromIndex,
 int bToIndex)
```

Compares two `Object` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing two elements of type `T` at a relative index `i` within the respective arrays that is the prefix length, as if by:

```
    Comparator.nullsFirst(Comparator.<T>naturalOrder()).
        compare(a[aFromIndex + i, b[bFromIndex + i])
```

Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(Object[], int, int, Object[], int, int)` for the definition of a common and proper prefix.)

The comparison is consistent with `equals`, more specifically the following holds for arrays `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively:

```
    Arrays.equals(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) ==
        (Arrays.compare(a, aFromIndex, aToIndex, b, bFromIndex, bToIndex) == 0)
```

API Note:

This method behaves as if (for non-`null` array elements):

```
  int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                          b, bFromIndex, bToIndex);
  if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
      return a[aFromIndex + i].compareTo(b[bFromIndex + i]);
  return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Type Parameters:

`T` - the type of comparable array elements

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if,

over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

compare

```
public static <T> int compare(T[] a,
                              T[] b,
                              Comparator<? super T> cmp)
```

Compares two `Object` arrays lexicographically using a specified comparator.

If the two arrays share a common prefix then the lexicographic comparison is the result of comparing with the specified comparator two elements at an index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two array lengths. (See `mismatch(Object[], Object[])` for the definition of a common and proper prefix.)

A `null` array reference is considered lexicographically less than a non-`null` array reference. Two `null` array references are considered equal.

API Note:

This method behaves as if (for non-`null` array references):

```
int i = Arrays.mismatch(a, b, cmp);
if (i >= 0 && i < Math.min(a.length, b.length))
    return cmp.compare(a[i], b[i]);
return a.length - b.length;
```

Type Parameters:

`T` - the type of array elements

Parameters:

`a` - the first array to compare

`b` - the second array to compare

`cmp` - the comparator to compare array elements

Returns:

the value `0` if the first and second array are equal and contain the same elements in the same order; a value less than `0` if the first array is lexicographically less than the second array; and a value greater than `0` if the first array is lexicographically greater than the second array

Throws:

`NullPointerException` - if the comparator is `null`

Since:

9

compare

```
public static <T> int compare(T[] a,
                              int aFromIndex,
                              int aToIndex,
                              T[] b,
                              int bFromIndex,
                              int bToIndex,
                              Comparator<? super T> cmp)
```

Compares two `Object` arrays lexicographically over the specified ranges.

If the two arrays, over the specified ranges, share a common prefix then the lexicographic comparison is the result of comparing with the specified comparator two elements at a relative index within the respective arrays that is the prefix length. Otherwise, one array is a proper prefix of the other and, lexicographic comparison is the result of comparing the two range lengths. (See `mismatch(Object[], int, int, Object[], int, int)` for the definition of a common and proper prefix.)

API Note:

This method behaves as if (for non-`null` array elements):

```
    int i = Arrays.mismatch(a, aFromIndex, aToIndex,
                            b, bFromIndex, bToIndex, cmp);
    if (i >= 0 && i < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
        return cmp.compare(a[aFromIndex + i], b[bFromIndex + i]);
    return (aToIndex - aFromIndex) - (bToIndex - bFromIndex);
```

Type Parameters:

`T` - the type of array elements

Parameters:

`a` - the first array to compare

`aFromIndex` - the index (inclusive) of the first element in the first array to be compared

`aToIndex` - the index (exclusive) of the last element in the first array to be compared

`b` - the second array to compare

`bFromIndex` - the index (inclusive) of the first element in the second array to be compared

`bToIndex` - the index (exclusive) of the last element in the second array to be compared

`cmp` - the comparator to compare array elements

Returns:

the value `0` if, over the specified ranges, the first and second array are equal and contain the same elements in the same order; a value less than `0` if, over the specified ranges, the first array is lexicographically less than the second array; and a value greater than `0` if, over the specified ranges, the first array is lexicographically greater than the second array

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array or the comparator is `null`

Since:

9

mismatch

```
public static int mismatch(boolean[] a,
                           boolean[] b)
```

Finds and returns the index of the first mismatch between two `boolean` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array

and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    a[pl] != b[pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(boolean[] a,
                           int aFromIndex,
                           int aToIndex,
                           boolean[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two `boolean` arrays over the specified ranges, otherwise return -1 if no mismatch

is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    a[aFromIndex + pl] != b[bFromIndex + pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(byte[] a,
                           byte[] b)
```

Finds and returns the index of the first mismatch between two `byte` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    a[pl] != b[pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

a - the first array to be tested for a mismatch

b - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise -1.

Throws:

NullPointerException - if either array is null

Since:

9

mismatch

```
public static int mismatch(byte[] a,
                           int aFromIndex,
                           int aToIndex,
                           byte[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two byte arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-null arrays, a and b with specified ranges [aFromIndex, aToIndex) and [bFromIndex, bToIndex) respectively, share a common prefix of length pl if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
```

```
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    a[aFromIndex + pl] != b[bFromIndex + pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(char[] a,
                           char[] b)
```

Finds and returns the index of the first mismatch between two `char` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    a[pl] != b[pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(char[] a,
                           int aFromIndex,
                           int aToIndex,
                           char[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two `char` arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    a[aFromIndex + pl] != b[bFromIndex + pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(short[] a,
                           short[] b)
```

Finds and returns the index of the first mismatch between two `short` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    a[pl] != b[pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(short[] a,
                           int aFromIndex,
                           int aToIndex,
                           short[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two `short` arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the

specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    a[aFromIndex + pl] != b[bFromIndex + pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(int[] a,
                           int[] b)
```

Finds and returns the index of the first mismatch between two `int` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    a[pl] != b[pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(int[] a,
                           int aFromIndex,
                           int aToIndex,
                           int[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two `int` arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    a[aFromIndex + pl] != b[bFromIndex + pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(long[] a,
                           long[] b)
```

Finds and returns the index of the first mismatch between two `long` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array

and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    a[pl] != b[pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(long[] a,
                           int aFromIndex,
                           int aToIndex,
                           long[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two `long` arrays over the specified ranges, otherwise return -1 if no mismatch is found.

The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    a[aFromIndex + pl] != b[bFromIndex + pl]
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(float[] a,
                           float[] b)
```

Finds and returns the index of the first mismatch between two `float` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    Float.compare(a[pl], b[pl]) != 0
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

a - the first array to be tested for a mismatch

b - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise -1.

Throws:

NullPointerException - if either array is null

Since:

9

mismatch

```
public static int mismatch(float[] a,
                           int aFromIndex,
                           int aToIndex,
                           float[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two float arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-null arrays, a and b with specified ranges [aFromIndex, aToIndex) and [bFromIndex, bToIndex) respectively, share a common prefix of length pl if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
```

```
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    Float.compare(a[aFromIndex + pl], b[bFromIndex + pl]) != 0
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(double[] a,
                           double[] b)
```

Finds and returns the index of the first mismatch between two `double` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    Double.compare(a[pl], b[pl]) != 0
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(double[] a,
                           int aFromIndex,
                           int aToIndex,
                           double[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two `double` arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    Double.compare(a[aFromIndex + pl], b[bFromIndex + pl]) != 0
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(Object[] a,
                           Object[] b)
```

Finds and returns the index of the first mismatch between two `Object` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl) &&
    !Objects.equals(a[pl], b[pl])
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length))
```

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static int mismatch(Object[] a,
                           int aFromIndex,
                           int aToIndex,
                           Object[] b,
                           int bFromIndex,
                           int bToIndex)
```

Finds and returns the relative index of the first mismatch between two `Object` arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the

specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    !Objects.equals(a[aFromIndex + pl], b[bFromIndex + pl])
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex))
```

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array is `null`

Since:

9

mismatch

```
public static <T> int mismatch(T[] a,
                               T[] b,
                               Comparator<? super T> cmp)
```

Finds and returns the index of the first mismatch between two `Object` arrays, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller array.

The specified comparator is used to determine if two array elements from the each array are not equal.

If the two arrays share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two elements at that index within the respective arrays. If one array is a proper prefix of the other then the returned index is the length of the smaller array and it follows that the index is only valid for the larger array. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b`, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(a.length, b.length) &&
    Arrays.equals(a, 0, pl, b, 0, pl, cmp)
    cmp.compare(a[pl], b[pl]) != 0
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b`, share a proper prefix if the following expression is true:

```
    a.length != b.length &&
    Arrays.equals(a, 0, Math.min(a.length, b.length),
                  b, 0, Math.min(a.length, b.length),
                  cmp)
```

Type Parameters:

`T` - the type of array elements

Parameters:

`a` - the first array to be tested for a mismatch

`b` - the second array to be tested for a mismatch

`cmp` - the comparator to compare array elements

Returns:

the index of the first mismatch between the two arrays, otherwise `-1`.

Throws:

`NullPointerException` - if either array or the comparator is `null`

Since:

9

mismatch

```
public static <T> int mismatch(T[] a,
                               int aFromIndex,
                               int aToIndex,
                               T[] b,
                               int bFromIndex,
                               int bToIndex,
                               Comparator<? super T> cmp)
```

Finds and returns the relative index of the first mismatch between two `Object` arrays over the specified ranges, otherwise return -1 if no mismatch is found. The index will be in the range of 0 (inclusive) up to the length (inclusive) of the smaller range.

If the two arrays, over the specified ranges, share a common prefix then the returned relative index is the length of the common prefix and it follows that there is a mismatch between the two elements at that relative index within the respective arrays. If one array is a proper prefix of the other, over the specified ranges, then the returned relative index is the length of the smaller range and it follows that the relative index is only valid for the array with the larger range. Otherwise, there is no mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a common prefix of length `pl` if the following expression is true:

```
    pl >= 0 &&
    pl < Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex) &&
```

```
    Arrays.equals(a, aFromIndex, aFromIndex + pl, b, bFromIndex, bFromIndex + pl
    cmp.compare(a[aFromIndex + pl], b[bFromIndex + pl]) != 0
```

Note that a common prefix length of `0` indicates that the first elements from each array mismatch.

Two non-`null` arrays, `a` and `b` with specified ranges [`aFromIndex`, `aToIndex`) and [`bFromIndex`, `bToIndex`) respectively, share a proper prefix if the following expression is true:

```
    (aToIndex - aFromIndex) != (bToIndex - bFromIndex) &&
    Arrays.equals(a, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  b, 0, Math.min(aToIndex - aFromIndex, bToIndex - bFromIndex),
                  cmp)
```

Type Parameters:

`T` - the type of array elements

Parameters:

`a` - the first array to be tested for a mismatch

`aFromIndex` - the index (inclusive) of the first element in the first array to be tested

`aToIndex` - the index (exclusive) of the last element in the first array to be tested

`b` - the second array to be tested for a mismatch

`bFromIndex` - the index (inclusive) of the first element in the second array to be tested

`bToIndex` - the index (exclusive) of the last element in the second array to be tested

`cmp` - the comparator to compare array elements

Returns:

the relative index of the first mismatch between the two arrays over the specified ranges, otherwise `-1`.

Throws:

`IllegalArgumentException` - if `aFromIndex > aToIndex` or if `bFromIndex > bToIndex`

`ArrayIndexOutOfBoundsException` - if `aFromIndex < 0 or aToIndex > a.length` or if `bFromIndex < 0 or bToIndex > b.length`

`NullPointerException` - if either array or the comparator is `null`

Since:

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.